上海海洋大學

本科毕业论文（设计）

中文题目：基于 Java 的就业指导系统的设计与实现

外文题目：Design and Implementation of a Java-Based Career Guidance System

学 院：爱恩学院

班 级：信管 4 班

姓 名：黄心璇

学 号：2291406

指导教师：卢鹏

2026 年 3 月

摘　要

本文围绕“基于 Java 的就业指导系统的设计与实现”展开综合性研究，旨在解决大学生就业指导效率不足、个性化程度较低以及信息服务分散等现实问题。随着人工智能、大数据分析与知识图谱等技术的发展，就业指导系统逐渐从传统线下咨询转向智能化、平台化服务。通过对国内外相关研究的系统梳理可以发现，国外在深度学习模型构建、知识图谱推荐与就业预测等方面已取得较成熟成果，国内则在就业指导体系建设、就业行为分析与信息化平台开发方面形成了较为完整的研究路径，但在智能化程度、系统自动化决策和模型深度应用方面仍有改进空间。

基于此，本文围绕 Java 技术体系，设计并实现一个具有岗位信息管理、用户画像构建、个性化推荐和就业数据分析等功能的就业指导系统。系统通过数据库管理、后端服务设计与推荐模块构建，提升就业信息整合效率，并为学生提供更具针对性的求职支持。研究结果表明，该系统具有良好的可扩展性与应用价值，能够在一定程度上改善传统就业指导依赖人工咨询、信息处理效率低等问题。

本文的研究不仅丰富了就业指导系统的技术实现路径，也为高校就业管理服务的数字化、智能化转型提供了参考。未来系统可进一步引入深度推荐模型、语义分析技术，以提升系统的智能化水平。

关键词： 就业指导系统；个性化推荐；Java；就业信息管理；智能化服务

Abstract

This paper presents a comprehensive study on the design and implementation of a Java-based career guidance system, aiming to address the practical challenges of low efficiency in graduate career counseling, insufficient personalization, and fragmented information services. With the advancement of artificial intelligence, big data analytics, and knowledge graph technologies, career guidance systems have gradually shifted from traditional offline consulting toward intelligent and platform-based services. A systematic review of related research at home and abroad reveals that overseas studies have achieved relatively mature results in deep learning model construction, knowledge graph-based recommendation, and employment prediction, while domestic research has established a relatively complete path in career guidance system development, employment behavior analysis, and information platform building. However, there remains room for improvement in terms of intelligence level, automated decision-making, and in-depth model application.

Based on the above, this paper designs and implements a career guidance system built upon the Java technology stack, incorporating functions such as job information management, user profile construction, personalized recommendation, and employment data analysis. Through database management, back-end service design, and recommendation module construction, the system improves the efficiency of employment information integration and provides students with more targeted job-seeking support. The results demonstrate that the system has good scalability and application value, and can effectively alleviate the problems of traditional career guidance relying heavily on manual consultation and low information processing efficiency.

This research not only enriches the technical implementation path of career guidance systems, but also provides a reference for the digital and intelligent transformation of university employment management services. In the future, the system can further incorporate deep recommendation models and semantic analysis technologies to enhance its overall level of intelligence.

Keywords: career guidance system; personalized recommendation; Java; employment information management; intelligent service

目录

第一章 绪论 1
1.1 研究背景及意义 1
1.2 国内外研究现状 1
1.2.1 国外研究现状 1
1.2.2 国内研究现状 2
1.3 研究思路与方法 3
1.4 论文组织结构 3
1.5 本章小结 4
第二章 系统开发的技术基础 4
2.1 体系结构分析 4
2.2 系统采用的开发技术 5
2.2.1 Vue 概述 5
2.2.2 Spring Boot 与 MyBatis 概述 6
2.2.3 MySQL 数据库 7
2.3 本章小结 7
第三章 系统的需求分析 7
3.1 可行性分析 8
3.1.1 技术可行性 8
3.1.2 经济可行性 8
3.1.3 操作可行性 9
3.2 系统流程分析 9
3.3 系统功能模块分析 10
3.3.1 管理员管理模块 11
3.3.2 辅导员管理模块 12
3.3.3 学生管理模块 12
3.3.4 企业管理模块 12
3.4 数据库分析 13
3.4.1 实体分析 13
3.4.2 实体关系分析 14
3.5 系统安全及界面设计需求 16
3.5.1 系统的可靠性要求 16
3.5.2 系统的界面设计需求 16
3.6 本章小结 16
第四章 系统的设计 17

4.1 系统总体设计----17
4.1.1 系统总体框架设计---- 17
4.1.2 系统总体结构设计---- 18
4.1.3 系统功能设计---- 19
4.2 系统的详细设计----19
4.2.1 系统功能模块的设计---- 19
4.2.2 系统数据库的设计---- 23
4.2.3 AI 智能推荐与对话模块设计---- 30
4.3 本章小结----30
第五章 系统的实现与测试---- 30
5.1 系统功能模块的实现----30
5.1.1 系统登录注册模块的实现---- 30
5.1.2 管理员管理模块的实现---- 36
5.1.3 辅导员管理模块的实现---- 42
5.1.4 学生管理模块的实现---- 46
5.1.5 企业管理模块的实现---- 51
5.1.6 AI 推荐模块的实现---- 59
5.2 系统测试----60
5.2.1 系统测试环境---- 60
5.2.2 系统测试用例---- 61
5.2.3 系统测试结果---- 62
5.3 本章小结----63
第六章 结论与展望---- 64
参考文献---- 65

第一章 绪论

1.1 研究背景及意义

近年来，随着高校毕业生数量的持续增加，就业市场竞争愈加激烈，许多地区的就业结构性矛盾日益突出，给大学生的就业指导工作带来了前所未有的挑战。传统的就业指导方式主要依赖人工咨询、线下讲座以及信息平台的简单展示，然而，这些方法存在指导内容的同质化、岗位匹配度低、服务形式单一等问题，难以满足现代学生对个性化和精准化就业服务的需求。与此同时，信息技术的飞速发展，尤其是人工智能、大数据和信息系统的应用，为就业指导模式的转型提供了强有力的技术支持。当前，国外在深度学习推荐、知识图谱推理和就业预测模型的构建方面已有较为深入的研究，然而，国内高校在就业服务平台的智能化、个性化推荐与数据分析方面仍处于起步阶段，亟需借助技术手段提升服务质量。因此，构建一个基于 Java 的就业指导系统，结合岗位管理、用户画像和智能推荐功能，已成为解决这一问题的迫切需求。通过该系统，可以有效提升就业指导的精准性和效率，同时为高校就业服务的数字化转型提供可行路径，具有重要的理论价值和实际意义。

本研究不仅在技术层面填补了国内在就业指导系统智能化建设上的空白，也为高校就业服务体系的优化提供了新的思路。从理论上看，本研究丰富了就业指导系统的技术架构和模型研究，尤其是在智能推荐、数据挖掘等前沿领域的应用研究方面，具有较高的学术价值。同时，本研究通过引入用户画像构建、岗位信息结构化管理和推荐算法，为就业指导的技术路径提供了新的视角。从实际意义来看，该系统可以显著提高高校就业部门的工作效率，减少传统人工整理的工作量，提高服务响应速度；同时，系统通过构建学生画像并提供个性化岗位推荐，有效提升推荐的精准度，满足学生的多样化需求，最终提高就业服务的质量和匹配度。基于 Java 的系统架构不仅具有良好的稳定性，还具备高度的扩展性，为后续智能推荐算法和就业数据分析模型的进一步开发奠定了基础。

1.2 国内外研究现状

1.2.1 国外研究现状

国外关于就业指导系统的研究起步较早，尤其是在智能化和个性化方面形成了较为成熟的研究成果。随着人工智能、机器学习及大数据技术的迅速发展，相关研究逐渐聚焦于将智能算法应用于就业指导系统，推动了该领域的发展。

在个性化就业与职业指导系统方面，Xu Xiaoxia[1]提出了一种端到端的深度神经网络模

型，该模型从职业行为、能力、兴趣及创业意向等多个维度进行数据建模，旨在实现更加精准的职业匹配，为智能就业指导系统提供了理论支持。Hajar Majjate 等[2]则应用了多种机器学习算法进行学业与院校推荐，其中 Huber 回归器的表现优于其他算法，这一方法同样能够为就业推荐系统提供指导。在知识图谱的应用方面，Siyue Cha 与 Hong Li[3]基于知识图谱设计了就业指导系统，通过简历和学习记录的语义分析实现了岗位推荐与资源推送，进一步提升了智能推理水平。此外，Juan Sánchez 等[4]通过人工神经模糊推断系统（ANFIS）研究了农业就业结构，并为政府提供了数据驱动的政策建议；Huan Yu 等[5]提出了一种基于模糊层次聚类与语义特征提取的大数据挖掘算法，显著增强了就业文本数据分析能力。Yu Wang[6]在其研究中结合了协同过滤与内容推荐算法，实验结果表明，个性化推荐能够有效提高学生与岗位的匹配度，进而提升系统的使用满意度。

在就业指导体系的设计方面，Juan Chen[7]通过对 589 份大学生职业规划问卷的分析，提出了从机构设置、课程设计、师资力量及评价体系四个维度构建就业指导体系的建议，强调了就业服务体系的制度化与结构化。Peisheng Wu[8]则从职业院校的实践出发，提出应通过开设实用课程、加强校企合作以及引入技术平台等措施，来提升学生的就业能力，并为就业指导体系的完善提供了操作性强的路径。

与此同时，国外的研究也关注就业信息平台与综合服务系统的建设。Jun Zheng 等[9]较早提出了基于 ASP.NET 的就业服务系统，该系统整合了学校、企业、政府和学生的各类信息资源，实现了岗位信息的集中发布和管理，有效地解决了传统就业服务中信息分散的问题，为后续的网络化就业系统建设提供了宝贵的经验。

1.2.2 国内研究现状

国内关于大学生就业指导的研究主要集中在就业指导体系建设、就业问题成因分析以及信息化就业指导系统的开发三个方向。总体来看，国内的研究强调教育政策、学校管理与信息技术的融合，逐步推动就业指导从经验式向数据化、平台化方向发展。

在就业指导的内涵与体系研究方面，王一栋等[10]提出，职业生涯规划教育能够有效缓解大学生就业难题，并建议通过课程与教育机制的创新提升学生的职业定位能力。黄克强等[11]结合高职院校的背景，提出应构建学校、企业与社会协同的高质量就业支持体系，以适应新时代就业形势的变化。李青[12]则探讨了“一站式学生社区”在就业指导中的应用，强调通过资源整合来提升服务响应效率。朱红维[13]总结了当前大学生就业难问题的结构性成因，并提出了相应的对策，为完善就业指导体系提供了实践依据。

在就业行为与就业问题分析方面，郑露与吴迪[14]围绕“慢就业”现象展开研究，分析了就业观念、个人准备度与社会因素的关系，提出了针对性的解决策略。王洁与杨文金[15]则从系统的角度研究了“慢就业”的表现与应对路径。李佳莹等[16]关注到“00 后”毕业生的内卷心理和就业焦虑，强调高校在就业指导过程中应增加心理支持，并有效缓解学生压力。这

些研究揭示了当前大学生就业群体面临的典型问题，为就业干预措施的设计提供了理论支持。

在就业信息化与技术应用方面，国内的研究逐渐展现出系统化与智能化的趋势。周洪兴[17]探讨了人工智能智能体在就业指导精准服务中的应用，指出智能技术能够提升岗位推荐与数据分析的能力。倪前龙[18]基于内容过滤技术，研究了个性化就业指导方法，并为推荐算法在就业领域的实际应用提供了路径。黄金波等[19]设计了“T 型人才”简历管理与推荐系统，展示了智能推荐技术在高校就业管理中的应用潜力。王宁[20]采用 Bootstrap 框架构建了就业信息管理系统，李燕燕[21]基于“互联网+”平台设计了创业就业指导平台，白岚[22]则开发了基于 HTML5 的校友就业指导系统。胡思文[23]和戴琼洁[24]等提出了在网络课程与数字化校园背景下的就业指导信息系统方案，标志着国内就业服务信息化正在向多层次推进。

此外，部分研究聚焦于就业指导理论框架的构建。窦凯[25]从高职院校视角出发，提出了完善就业指导机制的建议；廖列营[26]基于系统论构建了就业指导工作体系框架；薛鑫[27]和梁诗宇[28]则分别从系统设计和 CRM 模型的角度，提出了就业指导系统建设的思路。

1.3 研究思路与方法

本文以应届生就业指导需求为出发点，遵循软件工程中"需求分析—系统设计—编码实现—测试验证"的标准开发流程，完成就业指导系统的设计与实现。

在需求分析阶段，对上海海洋大学的校情进行实地考察，结合应届生在求职过程中面临的实际问题，明确系统需要支持的用户角色与业务流程，并据此划分功能模块，同时完成数据库的实体分析与实体关系分析。

在系统设计阶段，采用前后端分离的 B/S 架构，将系统拆分为前端界面与后端服务两个独立工程。数据库设计方面，先识别核心业务对象，再确定表与表之间的关联关系，形成完整的数据库表结构。AI 推荐模块的设计以大语言模型的 API 调用为基础，将模型的对话能力嵌入岗位推荐与简历优化功能中。

在编码实现阶段，前端与后端同步推进，通过 RESTful API 接口进行联调对接，各功能模块按照设计文档逐步完成开发。

在测试阶段，采用黑盒测试方法，针对各功能模块编写测试用例，验证系统功能是否符合预期，并对测试结果进行记录与分析。

1.4 论文组织结构

本文共分为六章，各章的主要内容安排如下。

第一章为绪论，介绍了本课题的研究背景及意义，梳理了国内外在就业指导系统领域的研究现状，指出了现有研究的不足之处，并在此基础上明确了本文的研究内容与目标。

第二章为系统的技术分析，对系统开发所涉及的关键技术进行了介绍。通过对 C/S 与

B/S 两种架构的对比确定了系统的体系结构，并分别介绍了前端框架 Vue3、后端框架 Spring Boot 与 MyBatis-Plus、数据库 MySQL 的技术特点。

第三章为系统的需求分析，从应届生求职的实际痛点出发，梳理了系统的业务流程，明确了四类用户角色各自的功能模块，完成了数据库的实体分析与实体关系分析，并对系统的安全性与界面设计需求进行了说明。

第四章为系统的设计，在需求分析的基础上进行系统的总体设计和详细设计，包括系统总体框架设计、功能模块的详细设计、数据库表结构设计以及 AI 智能推荐与对话模块的设计。

第五章为系统的实现与测试，展示了各功能模块的实际运行效果，包括登录注册、管理员管理、辅导员管理、学生管理、企业管理以及 AI 推荐模块的实现，并对系统进行了功能测试，验证了系统的可用性与稳定性。

第六章为结论与展望，对全文的工作进行了总结，指出了系统目前存在的不足，并对后续的改进方向进行了展望。

1.5 本章小结

本章作为全文的开篇，从研究背景出发说明了开发就业指导系统的现实需要，随后对国内外的相关研究进行了梳理，明确了现有研究在智能化推荐与高校实际应用场景结合方面的不足。在此基础上，确定了本文的研究思路与方法，并对全文的章节安排进行了说明。

第二章 系统开发的技术基础

2.1 体系结构分析

软件体系结构是构建计算机软件实践的基础，主流结构主要分为 C/S 结构和 B/S 结构两类[29]，下面对二者的特点进行简要对比分析，并结合本系统的实际情况确定最终采用的架构方案。

C/S（Client/Server）即客户端/服务器模式，是一种典型的两层架构。C/S 架构属于胖客户端架构，在客户端上实现绝大多数的业务逻辑和界面展示，客户端部分需要承受较大的压力。一旦系统需求发生变化，客户端和服务端的应用程序都需要进行变更，给应用维护和升级带来了极大的不便[30]。这种模式响应速度快、交互体验好，但每次系统升级都需要同步更新所有客户端，维护成本较高，跨平台能力也相对较弱。

B/S（Browser/Server）即浏览器/服务器模式，用户只需通过浏览器即可访问系统，无须安装任何客户端软件，主要业务逻辑在服务器端实现，极少数事务逻辑在浏览器中实现，

减轻了客户端的压力，因此 B/S 架构也被称为瘦客户端[30]。与 C/S 相比，B/S 结构在部署和维护上更加轻便，系统升级只需在服务端进行，用户刷新浏览器即可使用最新版本，同时天然支持跨平台访问，对用户设备没有特殊要求。

从技术架构角度来看，B/S 模式通常采用三层体系结构，即表示层、业务逻辑层和数据访问层。表示层由运行在浏览器中的 HTML、CSS 和 JavaScript 构成，负责用户界面的呈现与交互；业务逻辑层部署在应用服务器上，承担核心的数据处理与业务规则实现；数据访问层则负责与数据库进行通信，完成数据的存取操作。三层架构将系统的不同职责进行了清晰划分，各层之间通过标准接口进行交互，降低了模块之间的耦合度，也使得系统在后续的功能扩展与维护中更加灵活[30]。

在 B/S 架构的基础上，当前主流的 Web 应用开发进一步采用了前后端分离的开发模式。所谓前后端分离，是指将前端界面与后端业务逻辑彻底拆分为两个独立的工程，前端通过 HTTP 请求调用后端提供的 RESTful API 接口，后端以 JSON 格式返回数据，二者之间不再存在页面渲染层面的耦合。这种开发模式的核心优势在于：前后端可以各自选择最适合的技术栈进行独立开发与部署，开发团队之间的协作效率更高；前端框架可以专注于用户体验的优化，后端框架则专注于业务逻辑与数据管理；同时，接口的标准化使得系统具备更好的可测试性与可维护性[31]。目前，Spring Boot + Vue 是业界最主流的前后端分离技术组合方案之一，本系统的开发也采用了这一技术路线。

本系统面向学生、企业、辅导员和管理员四类用户群体，使用场景分散，对跨平台访问的需求较高，同时考虑到开发周期与后期维护成本，最终选用 B/S 架构模式进行系统构建，并采用前后端分离的方式组织开发，前端与后端作为两个独立工程分别进行开发与部署。

2.2 系统采用的开发技术

2.2.1 Vue 概述

Vue.js 是一套用于构建用户界面的渐进式 JavaScript 框架，由尤雨溪于 2014 年发布，目前已发展至 Vue3 版本。与其他前端框架相比，Vue 的核心设计理念是"渐进式"，开发者可以只将其用于页面的某一部分，也可以结合生态工具构建完整的单页应用，上手门槛相对较低[32]。

Vue3 在 Vue2 的基础上引入了 Composition API，使得代码组织更加灵活，逻辑复用也更为方便。与此同时，Vue3 重写了虚拟 DOM 并支持 Tree-shaking，打包体积更小，运行速度更快，整体性能相比上一版本有明显提升[33]。

Vue 采用 MVVM（Model-View-ViewModel）架构模式。在该模式下，Model 层负责管理应用的业务数据，View 层负责界面的渲染与展示，而 ViewModel 层作为二者之间的

桥梁，通过双向数据绑定机制实现数据与视图的自动同步。当 Model 中的数据发生变化时，View 会自动更新；反过来，当用户在 View 上进行操作时，数据也会同步反映到 Model 中。这种机制使得开发者无需手动操作 DOM 元素，极大地简化了前端开发的复杂度[33]。

与 React 和 Angular 等主流前端框架相比，Vue 的学习曲线更为平缓，模板语法更接近传统的 HTML 写法，对初学者十分友好。同时，Vue 拥有完善的生态系统，包括用于路由管理的 Vue Router、用于全局状态管理的 Pinia（Vuex 的替代方案）以及用于构建工具链的 Vite 等，能够满足从简单页面到大型单页应用的各种开发需求。在国内的前端开发社区中，Vue 的使用率和关注度均处于领先地位[32]。

本系统前端采用 Vue3 框架，结合 Element Plus 组件库进行开发。Element Plus 是基于 Vue3 的桌面端 UI 组件库，提供了丰富的现成组件，能够有效提升开发效率，同时保证界面风格的统一规范。

2.2.2 Spring Boot 与 MyBatis 概述

Spring Boot 是基于 Spring 框架开发的快速开发脚手架，其核心理念是"约定优于配置"，开发者无需编写大量繁琐的 XML 配置文件，即可快速搭建起一个可运行的 Spring 应用[31]。Spring Boot 内嵌了 Tomcat 服务器，支持直接以 jar 包形式部署运行，极大简化了项目的部署流程，是目前 Java 后端开发领域应用最广泛的框架之一[34]。

Spring Boot 继承了 Spring 框架的两大核心机制：控制反转（IoC）和面向切面编程（AOP）。IoC 通过依赖注入的方式将对象的创建与管理交由框架统一处理，开发者只需通过注解声明依赖关系即可，无需手动实例化对象，从而实现了模块间的松耦合。AOP 则允许开发者将日志记录、事务管理、权限校验等横切关注点从业务逻辑中分离出来，以切面的形式统一管理，提升了代码的可维护性和复用性[35]。此外，Spring Boot 提供了强大的自动配置（Auto-Configuration）能力，能够根据项目中引入的依赖自动推断并加载相应的默认配置，开发者仅需在 application.yml 或 application.properties 文件中进行少量的自定义配置即可完成项目的搭建，显著降低了项目的初始化门槛。

MyBatis 是一款持久层框架，支持通过 XML 或注解的方式编写 SQL 语句，并将执行结果自动映射为 Java 对象，省去了大量手动处理结果集的工作[32]。与全自动的 ORM 框架相比，MyBatis 将 SQL 的控制权交还给开发者，在需要复杂查询或性能调优的场景下更加灵活。

MyBatis 的前身是 Apache 社区的开源项目 iBatis，于 2010 年正式更名为 MyBatis[36]。其核心工作原理是：开发者在 XML 映射文件或 Java 注解中编写 SQL 语句，MyBatis 在运行时将 SQL 与 Java 方法进行绑定，并通过内置的参数处理器和结果集映射器完成 Java 对象与数据库记录之间的自动转换。这种半自动化的 ORM 方式既保留了 SQL 的灵活性，又避免了原生 JDBC 代码中大量的样板代码，在需要编写复杂联表查询、动态条件

拼接等场景下具有显著优势[36]。

本系统后端采用 Spring Boot 3.3.0，运行环境为 JDK 17，结合 MyBatis-Plus 进行开发。MyBatis-Plus 是 MyBatis 的增强版本，在保留原有灵活性的基础上，对单表的增删改查操作进行了大幅简化，进一步提升了开发效率。

2.2.3 MySQL 数据库

MySQL 是目前最流行的开源关系型数据库管理系统之一，采用标准的 SQL 语言进行数据操作，具有性能稳定、使用成本低、社区资源丰富等特点，被广泛应用于各类 Web 应用系统的后台数据存储[37]。

MySQL 的主要优点体现在以下几个方面：体积小、速度快，适合部署在资源有限的服务器环境；InnoDB 存储引擎提供了事务支持与行级锁机制，能够较好地保证数据的一致性与完整性；与 Spring Boot、MyBatis 的整合也十分成熟，开发配置简便。

MySQL 支持多种存储引擎，其中 InnoDB 是当前的默认引擎，也是本系统所采用的存储引擎。InnoDB 遵循 ACID（原子性、一致性、隔离性、持久性）事务模型，能够在高并发读写场景下保证数据操作的可靠性。同时，InnoDB 支持外键约束，有助于在数据库层面维护表与表之间的参照完整性。在索引方面，MySQL 默认使用 B+ 树索引结构，能够高效地支持范围查询与等值查询，对于本系统中涉及的岗位搜索、学生信息检索等高频查询场景具有较好的性能表现[38]。此外，MySQL 提供了丰富的管理工具和可视化客户端（如 Navicat、MySQL Workbench 等），便于开发人员进行数据库的日常管理与调试操作。

综合以上特点，本系统选用 MySQL 作为后台数据库，负责存储学生、企业、岗位、应聘记录及推荐日志等全部业务数据。

2.3 本章小结

本章主要介绍了系统开发所涉及的相关技术。通过对 C/S 与 B/S 两种体系结构的对比，结合本系统的实际需求，最终确定采用 B/S 架构模式。在开发技术上，前端采用 Vue3 结合 Element Plus，后端采用 Spring Boot 3.3.0（JDK 17）结合 MyBatis-Plus，数据库选用 MySQL，三者共同构成了本系统的技术栈，为后续的系统设计与实现提供了基础支撑。

第三章 系统的需求分析

近年来，随着科技的进步与经济形势的变化，国内众多企业纷纷推行降本增效策略，压缩用人成本，在招聘上也越来越倾向于有经验的成熟人才。夹在中间的应届生处境愈发

尴尬，就业压力史无前例巨大。

市面上不乏各类求职招聘软件，如 Boss 直聘、51Job、应届生求职网等，但这些平台大多从企业视角出发，核心是帮企业更高效地筛到人，真正站在应届生角度，帮他们规划求职方向、优化简历、精准匹配岗位的工具却少之又少。与此同时，招聘信息铺天盖地、良莠不齐，信息差问题始终困扰着求职者——资源整合不到位，往往意味着错过一个又一个本来合适的机会。

基于以上两个痛点，就业指导系统应运而生。本系统的需求来源于对应届生求职困境的实际观察与思考，旨在通过系统化的功能设计，切实帮助应届生理清求职思路、提升求职效率。考虑到系统目前仍处于起步阶段，学生用户群体暂时以应届生为主，并以上海海洋大学作为初期测试用例。

3.1 可行性分析

在进行系统的详细需求分析之前，有必要先对项目的可行性进行评估，以确认系统开发在技术、经济和操作层面均具备实施条件。

3.1.1 技术可行性

本系统采用 B/S 架构，前端基于 Vue3 与 Element Plus，后端基于 Spring Boot 3.3.0 和 MyBatis-Plus，数据库选用 MySQL，均为当前主流且技术成熟的开源框架和工具。Vue3 拥有完善的中文文档与活跃的社区支持，Element Plus 提供了丰富的现成 UI 组件，能够满足系统各类页面的开发需求。Spring Boot 内嵌 Tomcat 服务器，支持快速搭建与部署，MyBatis-Plus 对常见的数据库操作进行了大幅简化，开发效率较高。MySQL 作为开源关系型数据库，性能稳定、使用成本低，与 Spring Boot 的整合也十分成熟。AI 推荐模块通过调用字节跳动 Doubao 大模型的 API 接口实现，无需自行训练模型，接入成本较低。综合来看，本系统所涉及的技术栈均有充足的学习资源和社区支持，技术风险可控，在技术层面具备可行性。

3.1.2 经济可行性

本系统开发过程中使用的软件工具和框架均为开源免费产品。前端开发使用 Visual Studio Code 编辑器，后端开发使用 IntelliJ IDEA 专业版，数据库使用 MySQL 社区版，版本管理使用 Git，以上工具均无需支付授权费用。系统运行阶段，Spring Boot 内嵌 Tomcat 服务器，无需额外购置独立的应用服务器。在硬件方面，开发和测试阶段仅需一台普通个人计算机即可完成，无需购置专用服务器设备。AI 推荐模块调用的 Doubao 大模型 API 提供了一定额度的免费调用量，在系统初期测试阶段不会产生额外费用。整体开

发成本较低，在经济层面具备可行性。

3.1.3 操作可行性

本系统基于 B/S 架构，用户只需通过浏览器即可访问，无需在本地安装任何客户端软件，操作门槛较低。系统界面采用 Element Plus 组件库统一设计，各页面风格一致、布局简洁，表单均提供实时校验与填写提示，降低了用户的使用难度。系统按照学生、企业、辅导员、管理员四类角色分别设计了独立的功能入口，不同角色登录后只展示与自身权限对应的功能菜单，避免了信息干扰。对于管理员而言，系统提供了完整的后台管理界面，可对用户信息、岗位信息、就业数据等进行统一管理，操作流程清晰。综合来看，本系统在操作层面具备可行性。

3.2 系统流程分析

根据对上海海洋大学校情的实地考察，结合应届生求职的实际需要，将就业系统流程细化分析，以便合理嵌入系统供后续使用。

系统涉及四类用户角色，分别是学生、企业、辅导员和管理员，各角色在登录系统后进入各自的功能入口，业务流程围绕"学生求职"这条主线展开，企业、辅导员和管理员在各自的职责范围内参与其中。

学生登录后，首先需要完善个人基本信息和简历，填写就业意向、技能、工作经历等内容。系统会根据这些信息为学生匹配合适的岗位并推送推荐列表，学生也可以主动搜索岗位、筛选条件，找到心仪的职位后直接投递简历。投递完成后，学生可实时追踪应聘进展，收到面试邀请后在线确认，整个求职流程在系统内闭环完成。此外，学生还可以通过 AI 对话获取个性化的岗位建议和简历优化意见，查看辅导员发布的就业指导帖子和公开课信息，进一步提升其求职竞争力。

企业登录后，完善单位基本信息和岗位详情，系统会将匹配的求职者推荐给企业方。企业可主动浏览求职者资料，也可以借助 AI 对话筛选合适人才，向目标学生发出简历查看邀请和面试邀请，并在系统内统一管理应聘进度。

辅导员主要承担就业指导与监督的职责。辅导员登录后，可以查看所管辖学生的整体求职情况，发布就业指导帖子和公开课，并对学生就业数据进行统计分析，为有针对性地开展就业指导工作提供数据支撑。

管理员负责系统的整体运维，包括对学生、企业、辅导员账号信息的统一管理，以及就业统计数据的汇总与维护，保障系统的正常运转。

综上所述，就业指导系统管理流程如图所示：

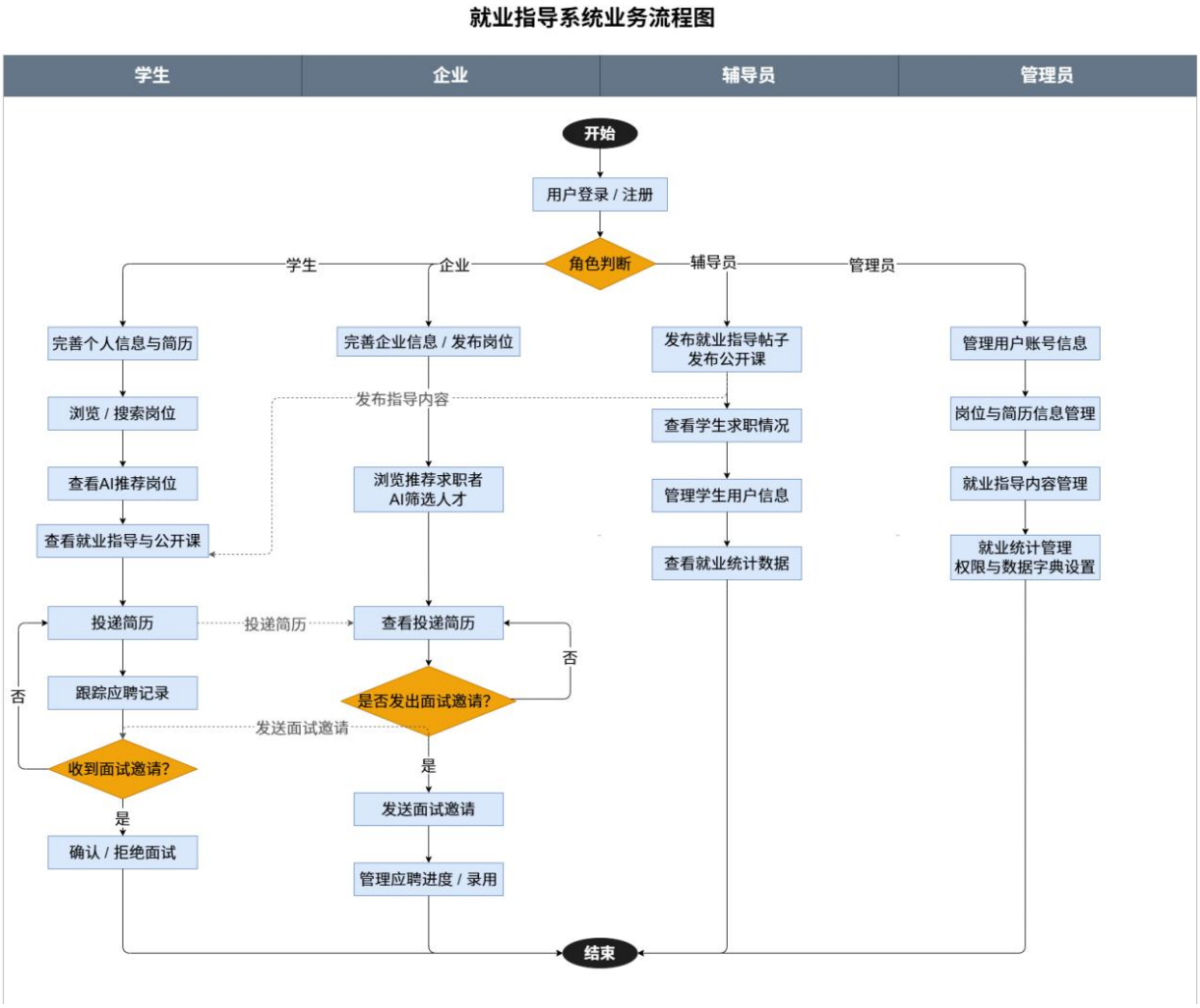


图 3.1 就业指导系统业务流程图

3.3 系统功能模块分析

如前所述，就业指导系统按需求分析，功能主要包括管理员管理模块、辅导员管理模块、学生管理模块、企业管理模块。

为了更直观地展示各角色与系统功能之间的关系，采用 UML 用例图对四类用户角色的功能需求进行建模，如图 3.3 所示。

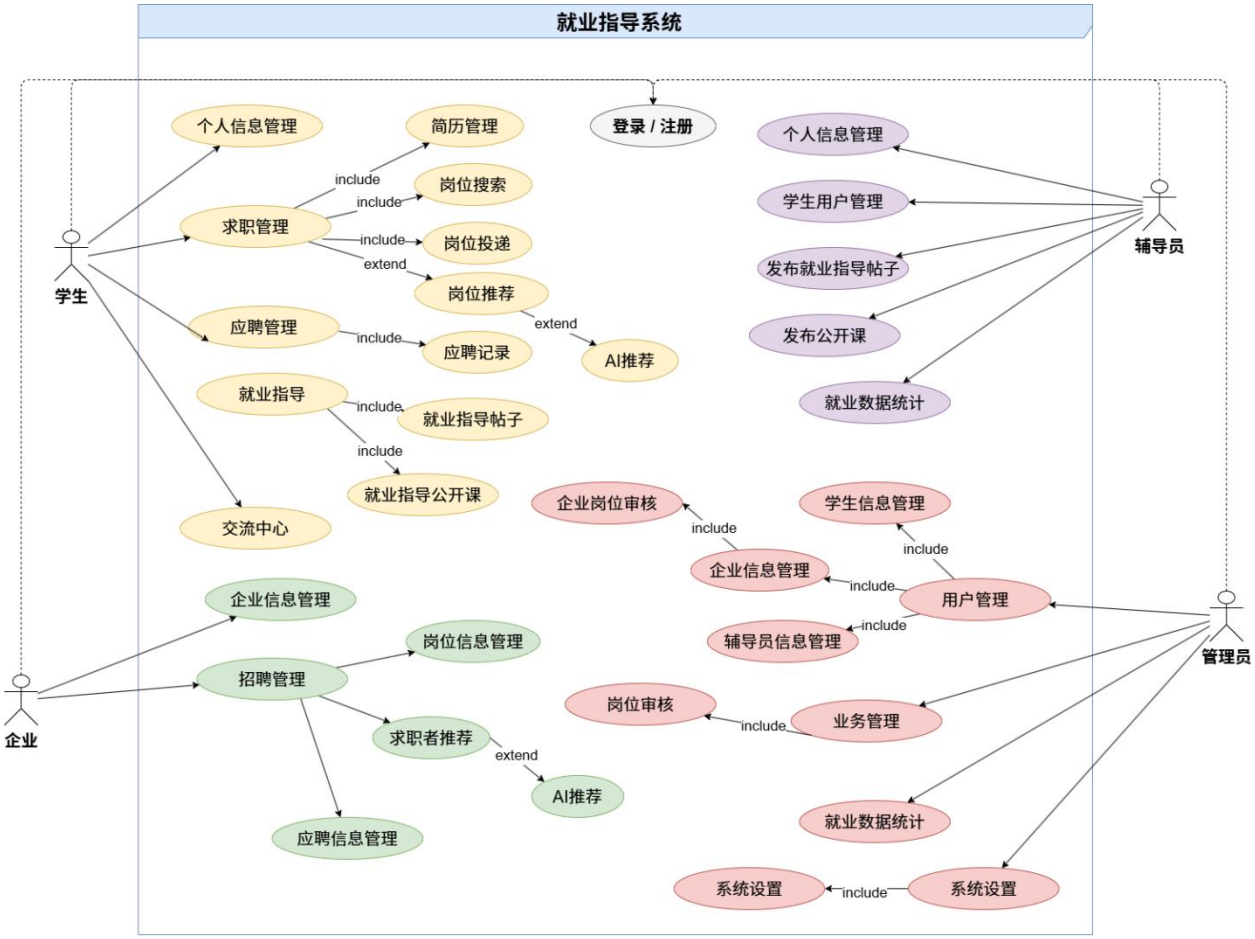


图 3.1 就业指导系统用例图

根据图 3.3，学生作为系统的核心用户，围绕求职这条主线与简历管理、岗位推荐、应聘记录追踪、就业指导、AI 推荐等用例产生交互；企业主要与岗位发布、求职者推荐、应聘管理等用例关联；辅导员侧重于就业指导内容的发布与学生就业数据的统计；管理员则拥有对系统内所有用户信息、岗位信息及基础数据的统一管理权限。各角色之间职责边界清晰，功能划分合理。

3.3.1 管理员管理模块

管理员作为整个系统的"管家"，拥有最高权限，负责维护系统的正常运转。管理员登录后可使用以下功能：

（1）个人信息管理：管理员可以查看和修改自身的账号基本信息。

（2）学生信息管理：对系统内所有学生用户的基本信息进行核实、查询与维护。

（3）企业信息管理：对已注册的企业信息进行统一审核与管理。

（4）辅导员信息管理：对辅导员账号信息进行管理，包括新增、修改、删除等操作。

（5）企业岗位管理：对企业发布的招聘岗位进行审核和管理，过滤不合规的岗位信息。

（6）简历信息管理：对学生提交的简历进行查看和统一管理。

（7）就业指导管理：对辅导员发布的就业指导帖子和公开课内容进行审核与管理。

（8）就业统计管理：汇总全校学生的就业情况数据，为学校就业工作的决策提供数据支撑。

（9）权限设置：为不同角色的用户分配相应的操作权限，保障系统的访问安全。

（10）数据字典：维护系统中各类基础数据的分类与编码，保证数据的一致性与规范性。

3.3.2 辅导员管理模块

辅导员主要承担对学生的就业指导与日常管理工作。辅导员登录后可使用以下功能：

（1）个人信息管理：辅导员可查看并修改自身的账号基本信息。

（2）学生用户管理：辅导员可对所管辖范围内的学生用户信息进行查看和管理。

（3）就业指导管理：辅导员可在系统内发布就业指导帖子和公开课，帮助学生了解求职技巧与行业动态。

（4）就业统计：辅导员可查看所管辖学生的就业统计数据，了解整体就业进展。

3.3.3 学生管理模块

学生是系统的核心用户群体，整个系统围绕学生求职这条主线设计。学生登录后可使用以下功能：

（1）个人信息管理：学生可填写和完善自身的基本信息，包括毕业院校、所学专业、就业意向城市、岗位意向等。

（2）简历管理：学生可在系统内创建和完善个人简历，并通过集成的 AI 对话功能获取简历优化建议。

（3）岗位推荐：系统通过标签匹配与协同过滤算法为学生推送匹配岗位，同时支持 AI 对话获取个性化岗位建议。

（4）我的应聘记录：学生可实时追踪每条投递的进展状态，并对企业发出的面试邀请进行在线确认或拒绝。

（5）就业指导：学生可查看辅导员发布的指导帖子，预约参加就业指导公开课，并通过 AI 进行求职咨询。

（6）交流中心：学生可在此与其他同学、辅导员进行交流互动，分享求职经验。

3.3.4 企业管理模块

企业作为系统的用人方，主要通过系统发布岗位、筛选人才和管理招聘流程。企业登

录后可使用以下功能：

（1）个人信息管理：企业可查看并完善自身的基本资料，包括公司介绍、规模、行业类别、联系方式等。

（2）岗位信息管理：企业可在系统内发布、编辑和管理招聘岗位信息，发布后的岗位将对学生端可见。

（3）求职者推荐：系统根据岗位信息自动推荐匹配求职者，企业也可通过 AI 对话进一步筛选合适人才。

（4）应聘信息管理：企业可统一管理所有投递该企业岗位的学生信息，查看简历并发出面试邀请。

3.4 数据库分析

数据库设计是系统开发的基础环节，其核心目标在于为给定的应用环境构造最优的数据库模式，保证数据的完整性与一致性，并有效提升系统的查询性能与可维护性[39]。本系统采用 MySQL 作为关系型数据库，结合系统的功能需求与用户角色划分，对数据库进行系统性的分析与设计。

3.4.1 实体分析

根据第三章需求分析的结果，本系统共涉及四类用户角色与多个核心业务对象，经整理后识别出以下主要实体：

（1）学生（student）

学生是系统的核心用户之一，其信息包括登录账号、真实姓名、所在院系、专业、求职意向、意向城市、联系方式及所属辅导员等。学生可以管理个人简历、浏览并投递岗位，同时接收系统的智能推荐结果。

（2）辅导员（counselor）

辅导员负责对学生的就业情况进行跟踪与指导，其信息包括登录账号、所属院系、联系方式及邮箱等。辅导员可发布就业指导帖子与公开课程，并管理其名下学生。

（3）企业（company）

企业用户在系统中注册后可发布招聘岗位。企业信息包括企业名称、所属行业、规模、地址及联系方式等。企业账号需经管理员审核通过后方可使用。

（4）管理员（admin）

管理员负责系统的整体运营管理，包括对学生、辅导员、企业账号的审核与管理，以及对数据字典的维护。管理员信息包括登录账号、密码及角色权限等。

（5）简历（resume）

简历隶属于学生实体，记录学生的教育背景、工作经历、技能标签等信息，是学生投

递岗位时的凭据，同时也是 AI 推荐引擎进行简历分析的数据来源。

（6）岗位（job）

岗位由企业发布，包含岗位名称、薪资范围、工作城市、岗位要求及状态等信息。岗位信息是推荐算法的核心匹配对象。

（7）应聘记录（application）

应聘记录记录学生对特定岗位的投递行为，关联学生、岗位与简历三个实体，并追踪投递状态与面试时间。

（8）推荐日志（recommendation_log）

推荐日志是本系统的特色实体，用于记录系统向学生推送岗位的过程数据，包括推荐评分、推荐理由、推荐类型、是否被点击及是否已投递等字段，为后续推荐算法的效果评估与优化提供数据支撑。

（9）指导帖子（guidance_post）

指导帖子由辅导员发布，包含标题与正文内容，面向学生提供就业政策解读、求职技巧等指导性信息。

（10）公开课（course）

公开课由辅导员创建，设有报名人数上限与开课时间，学生可通过课程报名表进行选课。

（11）课程报名（course_enrollment）

课程报名作为学生与公开课之间的关联实体，记录学生的选课行为与报名时间。

（12）交流帖子（forum_post）

交流帖子由学生发布，用于在系统内进行求职经验分享与交流，包含标题、正文等信息。

（13）数据字典（data_dict）

数据字典用于统一管理系统中各类枚举值与状态码，如岗位状态、审核状态、推荐类型等，便于系统的统一维护与扩展。

3.4.2 实体关系分析

根据上述实体的业务语义，各实体之间的关系分析如下：

一名辅导员可以管理多名学生，一名学生只归属于一名辅导员，二者之间为一对多关系。一名学生拥有一份简历，二者之间为一对一关系。一名学生可以发起多条应聘记录，一个岗位也可以收到多名学生的投递，应聘记录作为学生与岗位之间的关联实体，分别与二者构成多对一关系。一份简历可以被用于多次投递，简历与应聘记录之间同样为一对多关系。

一家企业可以发布多个岗位，企业与岗位之间为一对多关系。一名辅导员可以发布多

篇指导帖子，也可以创建多门公开课，辅导员与指导帖子、辅导员与公开课之间均为一对多关系。一门公开课可以被多名学生报名，一名学生也可以报名多门课程，课程报名作为二者之间的关联实体分别与其构成多对一关系。一名学生可以发布多篇交流帖子，学生与交流帖子之间为一对多关系。

在推荐模块中，系统会为每名学生针对不同岗位生成推荐记录，一名学生可以对应多条推荐日志，一个岗位也可以出现在多名学生的推荐列表中，推荐日志与学生、推荐日志与岗位之间均为多对一关系。

根据上述分析，本系统的数据库实体关系如图 3.3.2 所示。

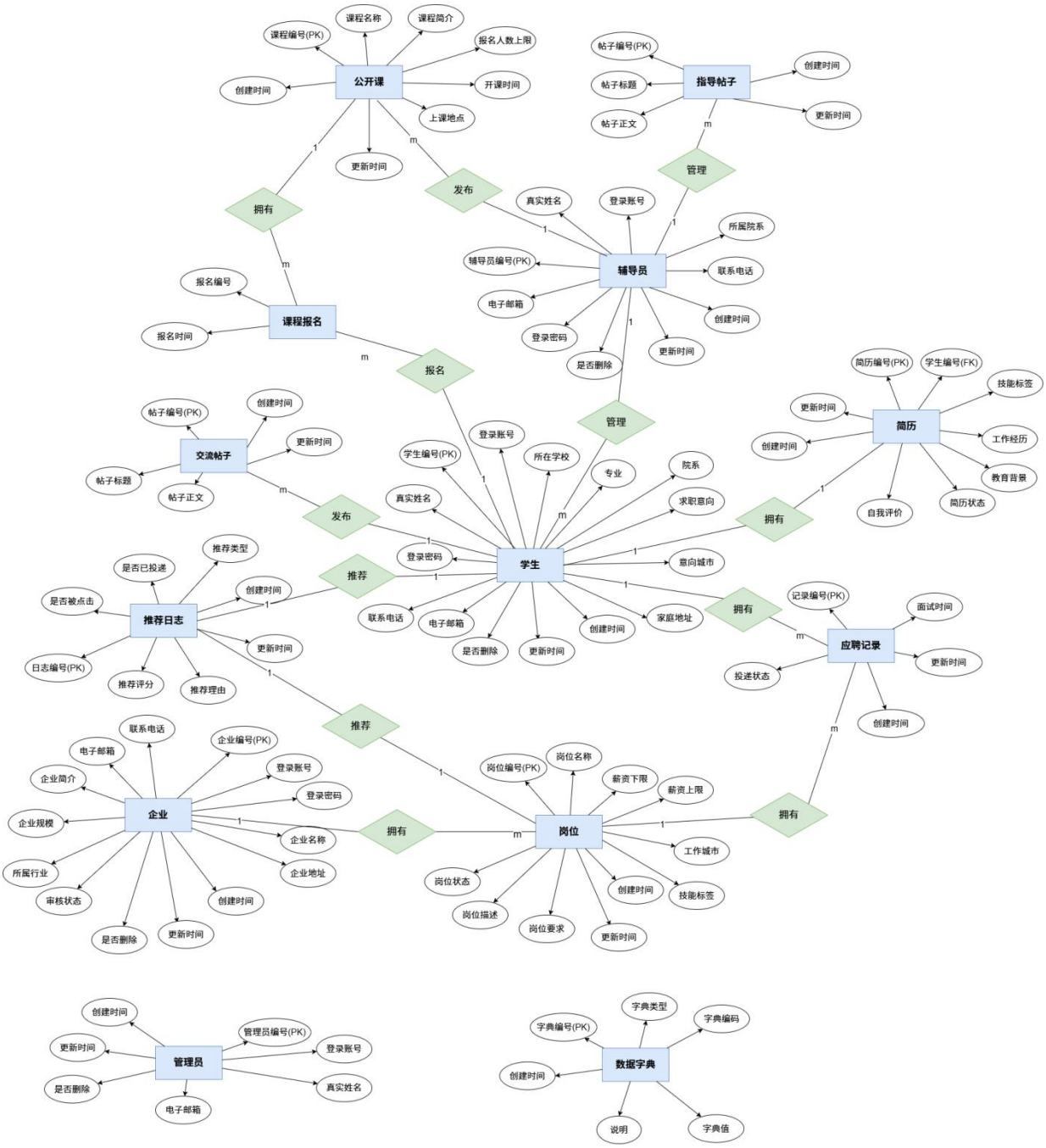


图 3.3.2 就业指导系统实体关系图（ERD）

3.5 系统安全及界面设计需求

3.5.1 系统的可靠性要求

本系统涉及学生个人信息、企业招聘数据及应聘记录等敏感数据，对系统的安全可靠性有一定要求，具体如下：

（1）系统采用基于 JWT（JSON Web Token）的身份认证机制。用户登录时须通过用户名与密码验证，验证通过后由服务端签发 Token，后续每次请求均需携带有效 Token 方可访问受保护资源。这防止了未授权用户非法访问系统数据的情况。

（2）系统对学生、企业、辅导员、管理员四类角色分别实施权限控制，不同角色只能操作与自身权限匹配的功能模块，越权访问将被系统拦截。

（3）用户登录密码禁止以明文形式存储，系统须对密码进行加密处理后方可写入数据库，保障用户账号安全。

（4）系统基于 B/S 架构部署，核心数据集中存储于服务端，客户端设备的损坏或丢失不会影响系统数据的完整性。

3.5.2 系统的界面设计需求

界面设计的好坏直接决定了用户对系统的使用体验。本系统在界面设计上需满足以下要求：

（1）系统采用 Vue3 框架结合 Element Plus 组件库进行前端开发，各页面字体、配色、按钮样式保持统一，整体风格简洁清爽。

（2）不同角色用户登录后只展示各自对应的功能菜单，无关功能不对其展示，减少信息干扰。

（3）系统支持主流浏览器访问，无需安装任何客户端，打开浏览器即可使用全部功能。

（4）表单须提供必要的填写提示与实时校验反馈，用户填写有误时能及时收到提示，避免操作失误。

3.6 本章小结

本章围绕就业指导系统的需求分析展开。首先对系统的技术可行性、经济可行性和操作可行性进行了论证，确认了系统开发的实施条件。随后对系统的业务流程进行了梳理，明确了学生、企业、辅导员和管理员四类用户角色各自的操作路径。在功能模块分析中，

通过 UML 用例图直观展示了各角色与系统功能之间的关系，并对各角色的功能模块进行了详细说明。此后从实体识别与关系梳理两个角度完成了数据库分析，给出了系统的 ER 图，并对系统的安全可靠性与界面设计需求作出了说明。

第四章 系统的设计

4.1 系统总体设计

4.1.1 系统总体框架设计

通过第三章对系统的需求分析，结合应届生求职的实际场景与各类用户角色的具体需求，就业指导系统的功能重点在于帮助学生高效匹配岗位、提升求职竞争力，同时为企业、辅导员和管理员提供各自所需的管理入口。系统按照四类用户角色划分功能模块，管理员负责系统的整体运维与数据管理，辅导员承担就业指导与学生监督的职责，学生围绕求职这条主线使用系统的核心功能，企业则通过系统完成岗位发布与招聘管理。根据以上功能划分，系统总体框架如图 4.1.1 所示。

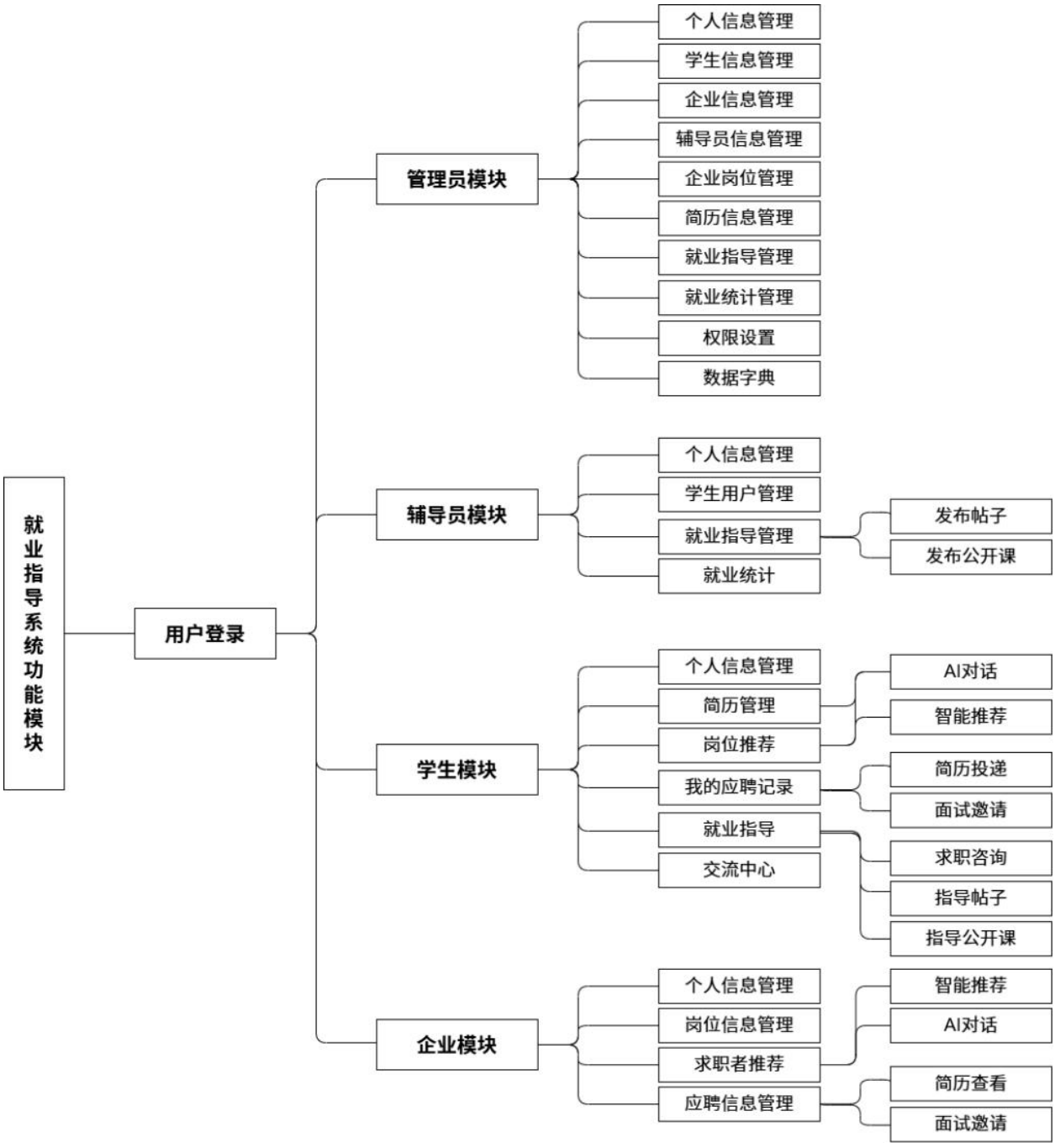


图 4.1.1 就业指导系统框架模块图

4.1.2 系统总体结构设计

本系统面向的用户群体较为分散，学生、企业、辅导员和管理员可能在不同地点、通过不同设备访问系统，既要保证信息传递的及时性与准确性，又要做到易于维护和扩展。综合考虑以上因素，系统最终选用 B/S 架构模式进行构建。用户只需通过浏览器即可访问系统的全部功能，无需安装任何客户端软件，跨平台的兼容性也得到了较好的保障。与此同时，系统的升级与日常维护只需在服务端进行，不会对用户的正常使用造成任何影响，整体运维成本较低。

4.1.3 系统功能设计

通过第三章系统需求分析，就业指导系统分为四大核心模块：管理员模块、辅导员模块、学生模块、企业模块。如图 4.1.3 所示。

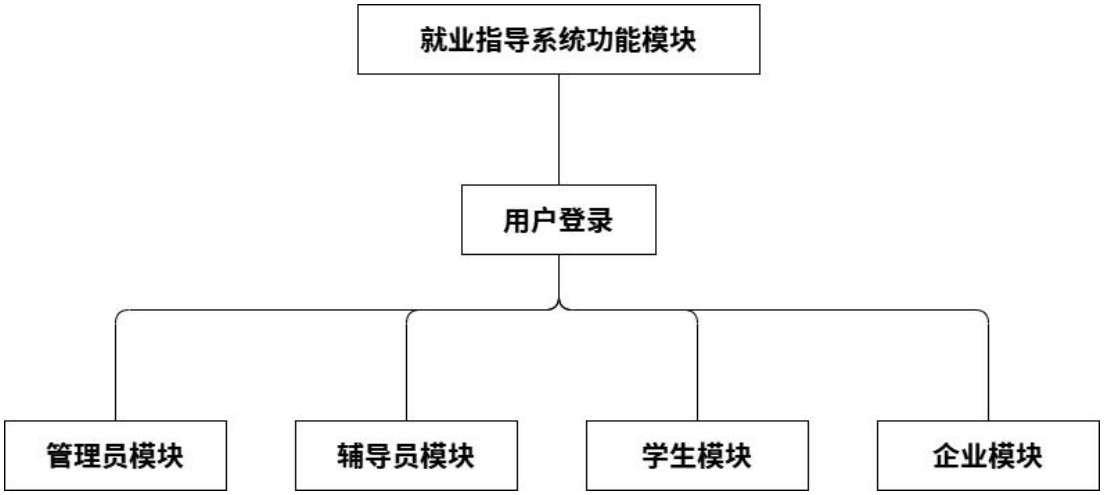


图 4.1.3 系统功能模块图

4.2 系统的详细设计

4.2.1 系统功能模块的设计

1. 管理员模块

管理员模块包括个人信息管理、学生信息管理、企业信息管理、辅导员信息管理、企业岗位管理、简历信息管理、就业指导管理、就业统计管理、权限设置、数据字典共 10 个子功能模块，如图 4.2.1-1 所示。

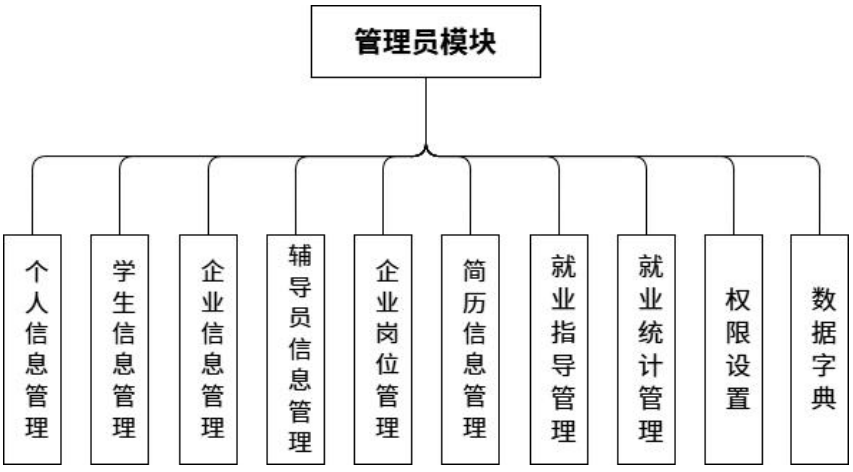


图 4.2.1-1 管理员功能模块

（1）个人信息管理：管理员可查看并修改自身的账号基本信息。

（2）学生信息管理：对系统内所有学生用户的基本信息进行核实、查询、编辑和维护，确保学生数据准确无误。

（3）企业信息管理：对已注册的企业信息进行统一审核与管理，保障入驻企业的信息真实可靠。

（4）辅导员信息管理：对辅导员账号信息进行管理，包括新增、修改、删除等操作。

（5）企业岗位管理：对企业发布的招聘岗位进行审核和管理，过滤不合规的岗位信息。

（6）简历信息管理：对学生提交的简历进行查看和统一管理。

（7）就业指导管理：对辅导员发布的就业指导帖子和公开课内容进行审核与管理。

（8）就业统计管理：汇总全校学生的就业情况数据，支持按院系、专业、班级等维度查询就业人数、就业去向及未就业情况，生成统计报表，为学校就业工作的决策及后续指导方向提供数据支撑。

（9）权限设置：为不同角色的用户分配相应的操作权限，保障系统的访问安全。

（10）数据字典：维护系统中各类基础数据的分类与编码，保证系统数据的一致性与规范性。

2. 辅导员模块

辅导员模块包括个人信息管理、学生用户管理、就业指导管理、就业统计共 4 个子功能模块，其中就业指导管理下设发布帖子与发布公开课两个子功能，如图 4.2.1-2 所示。

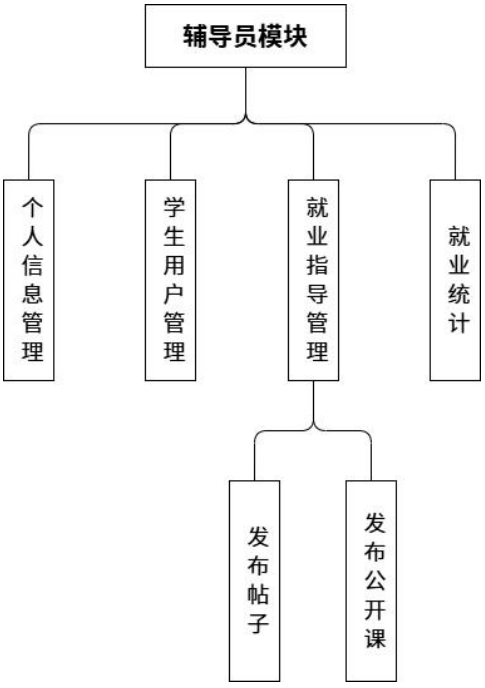


图 4.2.1-2 辅导员功能模块

（1）个人信息管理：辅导员可查看并修改自身的账号基本信息。

（2）学生用户管理：辅导员可对所管辖范围内的学生用户信息进行查看和管理，及时掌握学生的基本情况。

（3）就业指导管理：辅导员可在系统内发布就业指导相关内容，具体包括两个子功能：发布帖子，用于分享求职技巧、行业动态等文字性指导内容；发布公开课，用于创建线上或线下的就业指导课程，供学生预约参加。

（4）就业统计：辅导员可查看所管辖学生的整体就业统计数据，了解就业进展，从而有针对性地开展指导工作。

3. 学生模块

学生模块是整个系统功能最为丰富的部分，包括个人信息管理、简历管理、岗位推荐、我的应聘记录、就业指导、交流中心共 6 个子功能模块，如图 4.2.1-3 所示。

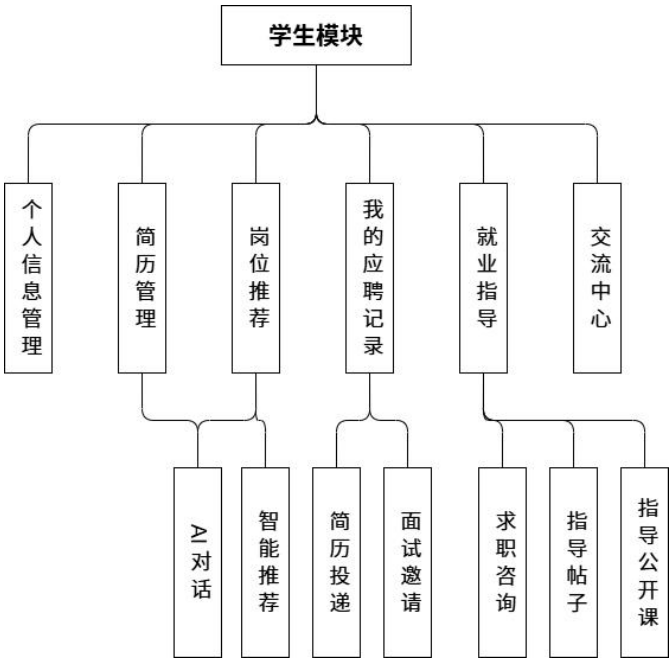


图 4.2.1-3 学生功能模块

（1）个人信息管理：学生可填写和完善自身的基本信息，包括毕业院校、所学专业、就业意向城市、岗位意向等，这些信息也是系统进行智能推荐的重要依据。

（2）简历管理：学生可在系统内创建和完善个人简历。模块内还集成了 AI 对话功能，学生可以直接与 AI 交流，获取简历优化建议，让简历更符合目标岗位的要求。

（3）岗位推荐：该模块下设智能推荐与 AI 对话两个子功能。智能推荐通过标签匹配与协同过滤算法为学生推送匹配度高的岗位列表；AI 对话则支持学生与 AI 进行一对一沟通，进一步获取个性化的岗位建议。

（4）我的应聘记录：该模块下设简历投递与面试邀请两个子功能。学生投递简历后

可在此实时追踪每条投递的进展状态；当企业发出面试邀请时，系统会及时提醒学生，并支持在线确认或拒绝，避免错过重要机会。

（5）就业指导：该模块下设求职咨询、指导帖子与指导公开课三个子功能，学生可通过 AI 获取求职建议，也可查看辅导员发布的指导内容，或预约参加就业指导课程，系统性地提升求职能力。

（6）交流中心：学生可在此与其他同学、辅导员进行交流互动，分享求职经验，互相答疑解惑。

4. 企业模块

企业模块包括个人信息管理、岗位信息管理、求职者推荐、应聘信息管理共 4 个子功能模块，其中求职者推荐下设 AI 对话与智能推荐，应聘信息管理下设简历查看与面试邀请，如图 4.2.1-4 所示。

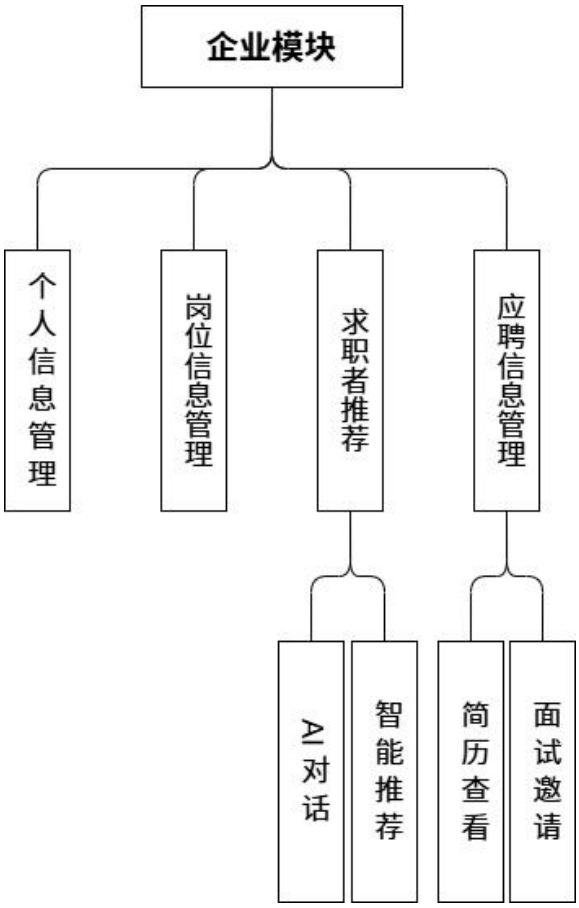


图 4.2.1-4 企业功能模块

（1）个人信息管理：企业可查看并完善自身的基本资料，包括公司介绍、规模、行业类别、联系方式等，完善的企业信息有助于吸引更多优质求职者。

（2）岗位信息管理：企业可在系统内发布、编辑和管理招聘岗位信息，包括岗位职责、任职要求、薪资待遇等详细内容，发布后的岗位将对学生端可见。

（3）求职者推荐：该模块下设智能推荐与 AI 对话两个子功能。系统会根据企业发布

的岗位信息自动推荐匹配度高的求职者；企业也可以通过 AI 对话描述用人需求，由 AI 协助进一步筛选合适人才，提升招聘效率。

（4）应聘信息管理：该模块下设简历查看与面试邀请两个子功能。企业可统一管理所有投递该企业岗位的学生信息，查看学生简历，并对感兴趣的学生发出面试邀请，实现招聘流程的闭环管理。

4.2.2 系统数据库的设计

根据前面系统功能模块的详细设计，所规划出的实体有：学生（student）、辅导员（counselor）、企业（company）、管理员（admin）、简历（resume）、岗位（job）、应聘记录（application）、推荐日志（recommendation_log）、指导帖子（guidance_post）、公开课（course）、课程报名（course_enrollment）、交流帖子（forum_post）、数据字典（data_dict）等等。

通过上面所列的实体规划，接下来就要把这些数据在概念上的结构通过 MySQL 数据库转换成能识别的逻辑结构[40]。本质就是根据上述实体以及实体之间的关系，最终形成对应的数据库表。

根据就业指导工作的特点，结合本系统的实际业务需要，本系统数据库主要设计了学生信息表、简历表、应聘记录表、企业信息表、岗位表、辅导员信息表、指导帖子表、公开课表、课程报名表、交流帖子表、推荐日志表、管理员信息表、数据字典表共 13 张数据表。各表结构如下：

（1）学生信息表（student）

学生信息表用于存储系统内所有学生用户的基本信息，包括登录账号、真实姓名、所在院系、专业、求职意向等，同时通过外键关联所属辅导员。结构如表 4.2.2-1 所示。

表 4.2.2-1 学生信息表（student）

字段名	数据类型	长度	允许为空	说明
id	int		否	主键，学生编号
user_name	varchar	50	否	登录账号
password	varchar	100	否	登录密码
real_name	varchar	50	是	真实姓名
school	varchar	100	是	所在学校
major	varchar	50	是	所学专业
department	varchar	50	是	所在院系
job_intention	varchar	100	是	求职意向
city_intention	varchar	50	是	意向城市

phone	varchar	20	是	联系电话
email	varchar	100	是	电子邮箱
address	varchar	200	是	家庭地址
counselor_id	int		是	外键，辅导员编号
create_at	datetime		是	创建时间
updated_at	datetime		是	更新时间
is_del	tinyint		否	是否删除

（2）简历表（resume）

简历表用于存储学生的个人简历信息，包括教育背景、工作经历、技能标签等内容，与学生信息表通过外键关联，是学生投递岗位时的凭据，也是推荐算法进行匹配分析的数据来源。结构如表 4.2.2-2 所示。

表 4.2.2-2 简历表（resume）

字段名	数据类型	长度	允许为空	说明
id	int		否	主键，简历编号
student_id	int		否	外键，学生编号
education	text		是	教育背景
experience	text		是	工作/实习经历
skills	varchar	500	是	技能标签
self_eval	text		是	自我评价
status	tinyint		是	简历状态
create_at	datetime		是	创建时间
updated_at	datetime		是	更新时间

（3）应聘记录表（application）

应聘记录表记录学生对特定岗位的投递行为，通过外键分别关联学生表、岗位表与简历表，并追踪投递状态与面试时间。结构如表 4.2.2-3 所示。

表 4.2.2-3 应聘记录表（application）

字段名	数据类型	长度	允许为空	说明
id	int		否	主键，记录编号

student_id	int		否	外键，学生编号
job_id	int		否	外键，岗位编号
resume_id	int		否	外键，简历编号
status	tinyint		是	投递状态
interview_time	datetime		是	面试时间
create_at	datetime		是	创建时间
updated_at	datetime		是	更新时间

（4）企业信息表（company）

企业信息表存储系统内所有企业用户的基本资料，包括登录账号、企业名称、所属行业、规模及联系方式等，企业账号需经管理员审核通过后方可正常使用。结构如表 4.2.2-4 所示。

表 4.2.2-4 企业信息表（company）

字段名	数据类型	长度	允许为空	说明
id	int		否	主键，企业编号
user_name	varchar	50	否	登录账号
password	varchar	100	否	登录密码
company_name	varchar	100	否	企业名称
industry	varchar	50	是	所属行业
scale	varchar	50	是	企业规模
address	varchar	200	是	企业地址
phone	varchar	20	是	联系电话
email	varchar	100	是	电子邮箱
description	text		是	企业简介
status	tinyint		是	审核状态
create_at	datetime		是	创建时间
updated_at	datetime		是	更新时间
is_del	tinyint		否	是否删除

（5）岗位表（job）

岗位表存储企业发布的招聘岗位信息，包括岗位名称、薪资范围、工作城市及岗位要求等，通过外键关联企业信息表，是推荐算法进行匹配的核心数据对象。结构如表 4.2.2-5 所示。

表 4.2.2-5 岗位表（job）

字段名	数据类型	长度	允许为空	说明
id	int		否	主键，岗位编号
company_id	int		否	外键，企业编号
job_name	varchar	100	否	岗位名称
salary_min	int		是	薪资下限
salary_max	int		是	薪资上限
city	varchar	50	是	工作城市
skills	varchar	500	是	技能标签
requirement	text		是	岗位要求
description	text		是	岗位描述
status	tinyint		是	岗位状态
create_at	datetime		是	创建时间
updated_at	datetime		是	更新时间

（6）辅导员信息表（counselor）

辅导员信息表存储系统内所有辅导员用户的基本信息，包括登录账号、所属院系及联系方式等，辅导员通过外键与其名下学生进行关联。结构如表 4.2.2-6 所示。

表 4.2.2-6 辅导员信息表（counselor）

字段名	数据类型	长度	允许为空	说明
id	int		否	主键，辅导员编号
user_name	varchar	50	否	登录账号
password	varchar	100	否	登录密码
real_name	varchar	50	是	真实姓名
department	varchar	50	是	所属院系
phone	varchar	20	是	联系电话
email	varchar	100	是	电子邮箱

create_at	datetime		是	创建时间
updated_at	datetime		是	更新时间
is_del	tinyint		否	是否删除

（7）指导帖子表（guidance_post）

指导帖子表存储辅导员发布的就业指导内容，包括帖子标题与正文，通过外键关联辅导员信息表。结构如表 4.2.2-7 所示。

表 4.2.2-7 指导帖子表（guidance_post）

字段名	数据类型	长度	允许为空	说明
id	int		否	主键，帖子编号
counselor_id	int		否	外键，辅导员编号
title	varchar	200	否	帖子标题
content	text		否	帖子正文
create_at	datetime		是	创建时间
updated_at	datetime		是	更新时间

（8）公开课表（course）

公开课表存储辅导员创建的就业指导课程信息，包括课程名称、开课时间、上课地点及报名人数上限等，通过外键关联辅导员信息表。结构如表 4.2.2-8 所示。

表 4.2.2-8 公开课表（course）

字段名	数据类型	长度	允许为空	说明
id	int		否	主键，课程编号
counselor_id	int		否	外键，辅导员编号
title	varchar	200	否	课程名称
description	text		是	课程简介
max_students	int		是	报名人数上限
course_time	datetime		是	开课时间
location	varchar	200	是	上课地点
create_at	datetime		是	创建时间
updated_at	datetime		是	更新时间

（9）课程报名表（course_enrollment）

课程报名表作为学生与公开课之间的关联实体，记录学生的选课行为与报名时间，通过外键分别关联学生表与公开课表。结构如表 4.2.2-9 所示。

表 4.2.2-9 课程报名表（course_enrollment）

字段名	数据类型	长度	允许为空	说明
id	int		否	主键，报名编号
student_id	int		否	外键，学生编号
course_id	int		否	外键，公开课编号
create_at	datetime		是	报名时间

（10）交流帖子表（forum_post）

交流帖子表存储学生在交流中心发布的帖子内容，包括帖子标题与正文，通过外键关联学生信息表。结构如表 4.2.2-10 所示。

表 4.2.2-10 交流帖子表（forum_post）

字段名	数据类型	长度	允许为空	说明
id	int		否	主键，帖子编号
student_id	int		否	外键，学生编号
title	varchar	200	否	帖子标题
content	text		否	帖子正文
create_at	datetime		是	创建时间
updated_at	datetime		是	更新时间

（11）推荐日志表（recommendation_log）

推荐日志表是本系统的特色数据表，用于记录系统向学生推送岗位的过程数据，包括推荐评分、推荐理由、推荐类型、是否被点击及是否已投递等字段，为后续推荐算法的效果评估与优化提供数据支撑。结构如表 4.2.2-11 所示。

表 4.2.2-11 推荐日志表（recommendation_log）

字段名	数据类型	长度	允许为空	说明
id	int		否	主键，日志编号
student_id	int		否	外键，学生编号
job_id	int		否	外键，岗位编号

score	float		是	推荐评分
reason	varchar	500	是	推荐理由
rec_type	tinyint		是	推荐类型
is_clicked	tinyint		是	是否被点击
is_applied	tinyint		是	是否已投递
create_at	datetime		是	创建时间
updated_at	datetime		是	更新时间

（12）管理员信息表（admin）

管理员信息表存储系统管理员的账号信息，包括登录账号、密码及联系方式等。结构如表 4.2.2-12 所示。

表 4.2.2-12 管理员信息表（admin）

字段名	数据类型	长度	允许为空	说明
id	int		否	主键，管理员编号
user_name	varchar	50	否	登录账号
password	varchar	100	否	登录密码
real_name	varchar	50	是	真实姓名
email	varchar	100	是	电子邮箱
create_at	datetime		是	创建时间
updated_at	datetime		是	更新时间
is_del	tinyint		否	是否删除

（13）数据字典表（data_dict）

数据字典表用于统一管理系统中各类枚举值与状态码，如岗位状态、审核状态、推荐类型等，便于系统的统一维护与扩展。结构如表 4.2.2-13 所示。

表 4.2.2-13 数据字典表（data_dict）

字段名	数据类型	长度	允许为空	说明
id	int		否	主键，字典编号
dict_type	varchar	100	否	字典类型
dict_code	varchar	100	否	字典编码

dict_value	varchar	200	否	字典值
description	varchar	200	是	说明
create_at	datetime		是	创建时间

4.2.3 AI 智能推荐与对话模块设计

AI 智能推荐与对话模块分别集成在学生端和企业端，底层统一接入字节跳动旗下的 Doubao-Seed-2.0-mini 大模型，通过调用其对话接口实现智能推荐与多轮问答功能。

对于学生用户，进入 AI 推荐模块后，系统会自动提取该学生 resume 表中的技能标签（skills）与自我评价（self_eval），结合 job 表中所有审核通过岗位的技能要求（skills）与岗位描述（description），构造提示词后调用大模型，推荐最合适的岗位并给出推荐理由。对于企业用户则相反，系统读取该企业在 job 表中已发布岗位的技能要求（skills）与岗位描述（description），结合系统内所有学生 resume 表中的技能标签（skills）与自我评价（self_eval），推荐符合条件的求职者。

两端都支持多轮对话，用户可以继续追问，例如补充筛选条件、询问简历怎么改、岗位信息怎么完善等，系统会把完整的对话历史传给模型，保持上下文连续。

4.3 本章小结

本章对就业指导系统进行了详细设计。总体设计部分确定了系统的整体框架、B/S 体系结构以及四大功能模块的划分。详细设计部分对管理员、辅导员、学生、企业四类角色的功能模块逐一进行了设计说明，完成了 13 张数据库表的结构设计与 ER 图绘制，并对 AI 智能推荐与对话模块的实现思路进行了阐述，为后续系统的开发实现提供了依据。

第五章 系统的实现与测试

5.1 系统功能模块的实现

5.1.1 系统登录注册模块的实现

（1）登录模块

系统登录模块是整个系统的入口，学生、企业、辅导员和管理员均通过统一的登录入口进行身份验证。用户输入账号与密码后，前端首先进行非空校验，校验通过后将请求发

送至后端，后端查询数据库并对密码进行验证，验证失败则返回错误提示，验证成功则根据用户角色签发对应的 JWT Token，前端接收 Token 后将其存储于本地，并根据角色类型自动跳转至对应的功能首页。登录模块的实现流程如图 5.1.1-1 所示，实现界面如图 5.1.1-2 所示。

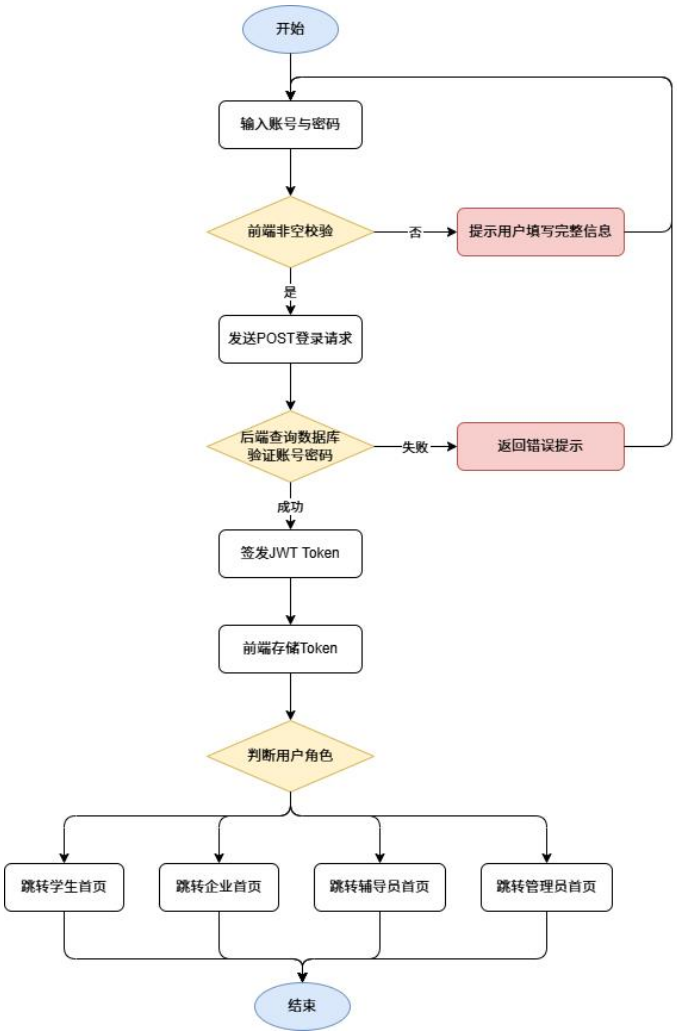


图 5.1.1-1 就业指导系统登录模块操作流程图



图 5.1.1-2 系统登录模块实现界面

登录模块实现的核心代码如下：

```
async function handleLogin() {
  if (locked.value) return
  const valid = await loginFormRef.value.validate().catch(() => false)
  if (!valid) return
  loading.value = true
  try {
    const res = await fetch('http://localhost:9090/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username: form.username, password: form.password })
    })
    const data = await res.json()
    if (data.code === 200) {
      localStorage.setItem('token', data.data.token)
      localStorage.setItem('role', data.data.role)
      localStorage.setItem('realName', data.data.realName || '')
      localStorage.setItem('userName', data.data.userName || '')
      localStorage.setItem('userId', data.data.userId || '')
      if (data.data.companyStatus !== undefined && data.data.companyStatus !== null) {
        localStorage.setItem('companyStatus', data.data.companyStatus)
      }
```

```
            ElMessage.success('登录成功')
            router.push('/')
        } else {
            if (data.message && data.message.startsWith('LOCKED:')) {
                locked.value = true
                ElMessage.error(data.message.replace('LOCKED:', ''))
            } else {
                ElMessage.error(data.message || '登录失败')
            }
        }
    } catch {
        ElMessage.error('网络异常，请检查后端服务是否启动')
    } finally {
        loading.value = false
    }
}
```

（2）注册模块

注册模块包括辅导员注册、学生注册和企业注册。当用户首次进入平台时，需要完成个人信息的注册，然后才可以用户名和密码登录。辅导员注册页面包括登录账号、密码、邮箱、手机号和所属院系。学生注册页面包括登录账号、密码、真实姓名、邮箱、手机号、所在学校、所在院系、所学专业和选择指导老师。指导老师选择为下拉框选择模式，数据来源于 counselor 表。企业注册页面包括登录账号、密码、真实姓名、邮箱、手机号、企业名称和所属行业。这里展示学生实现界面，示例如图 5.1.1-3 所示。



图 5.1.1-3 系统注册模块（学生）实现界面

注册模块实现的核心代码如下：

```
<el-form ref="formRef" :model="regForm" :rules="rules" label-width="0" class="login-form">
        <el-form-item prop="role">
          <el-select v-model="regForm.role" placeholder="请选择注册角色" style="width:
100%">
            <el-option label="学生" value="student" />
            <el-option label="辅导员" value="counselor" />
            <el-option label="企业" value="company" />
          </el-select>
        </el-form-item>
        <el-form-item prop="username">
          <el-input v-model="regForm.username" placeholder="登录账号
" :prefix-icon="User" />
        </el-form-item>
        <el-form-item prop="password">
          <el-input v-model="regForm.password" placeholder="密码（至少 6 位）
" :prefix-icon="Lock" show-password />
        </el-form-item>
        <el-form-item>
          <el-input v-model="regForm.realName" placeholder="真实姓名（选填）
" :prefix-icon="Avatar" />
        </el-form-item>
        <el-form-item>
          <el-input v-model="regForm.email" placeholder="邮箱（选填）
" :prefix-icon="Message" />
        </el-form-item>
        <el-form-item>
          <el-input v-model="regForm.phone" placeholder="手机号（选填）
" :prefix-icon="Phone" />
        </el-form-item>
        <!-- 学生字段 -->
        <template v-if="regForm.role === 'student'">
          <el-form-item>
            <el-input v-model="regForm.school" placeholder="所在学校（选填）
```

```
" :prefix-icon="School" />
            </el-form-item>
            <el-form-item>
              <el-input v-model="regForm.department" placeholder="所在院系（选填）
" :prefix-icon="OfficeBuilding" />
            </el-form-item>
            <el-form-item>
              <el-input v-model="regForm.major" placeholder="所学专业（选填）
" :prefix-icon="Reading" />
            </el-form-item>
            <el-form-item>
              <el-select v-model="regForm.counselorId" placeholder="选择指导老师（选填）
" style="width: 100%;" clearable>
                <el-option
                  v-for="c in counselorOptions" :key="c.id"
                  :label="c.realName + (c.department ? ' • ' + c.department : '')"
                  :value="c.id" />
              </el-select>
            </el-form-item>
          </template>
          <!-- 辅导员字段 -->
          <template v-if="regForm.role === 'counselor'">
            <el-form-item>
              <el-input v-model="regForm.department" placeholder="所属院系（选填）
" :prefix-icon="OfficeBuilding" />
            </el-form-item>
          </template>
          <!-- 企业字段 -->
          <template v-if="regForm.role === 'company'">
            <el-form-item prop="companyName">
              <el-input v-model="regForm.companyName" placeholder="企业名称（必填）
" :prefix-icon="OfficeBuilding" />
            </el-form-item>
            <el-form-item>
              <el-input v-model="regForm.industry" placeholder="所属行业（选填）
```

" :prefix-icon="Briefcase" />

</el-form-item>

</template>

<el-button class="login-btn" :loading="loading" @click="handleRegister">注 册 </el-button>

已有账号？<el-button type="primary" link @click="router.push('/login')">返回登录</el-button>

5.1.2 管理员管理模块的实现

管理员拥有系统最高权限，登录后可通过左侧导航栏进入首页、用户管理、业务管理、就业统计和系统设置五大功能模块。

（1）首页

管理员登录后进入系统首页，顶部四张卡片直观展示总学生数、总企业数、今日新增岗位数和待审核企业数。页面中部以条形图展示近 7 天每日新增学生趋势，下方列出最近注册企业名单，含注册时间与审核状态，管理员可在此直接跳转审核。实现界面如图 5.1.2-1 所示。



图 5.1.2-1 管理员页面首页实现界面

（2）用户管理页面

用户管理分为学生管理、企业管理和辅导员管理三个子页面。

学生管理和辅导员管理功能相近，均支持按姓名、院系等条件检索，以及对账号信息

进行新增、编辑和删除操作。实现界面如图 5.1.2-2、图 5.1.2-4 所示。

企业管理的核心是审核流程。管理员可对待审核企业执行通过、拒绝或删除，未通过的企业无法使用企业端功能；对于已通过的企业，管理员也可随时将其删除。实现界面如图 5.1.2-3 所示。

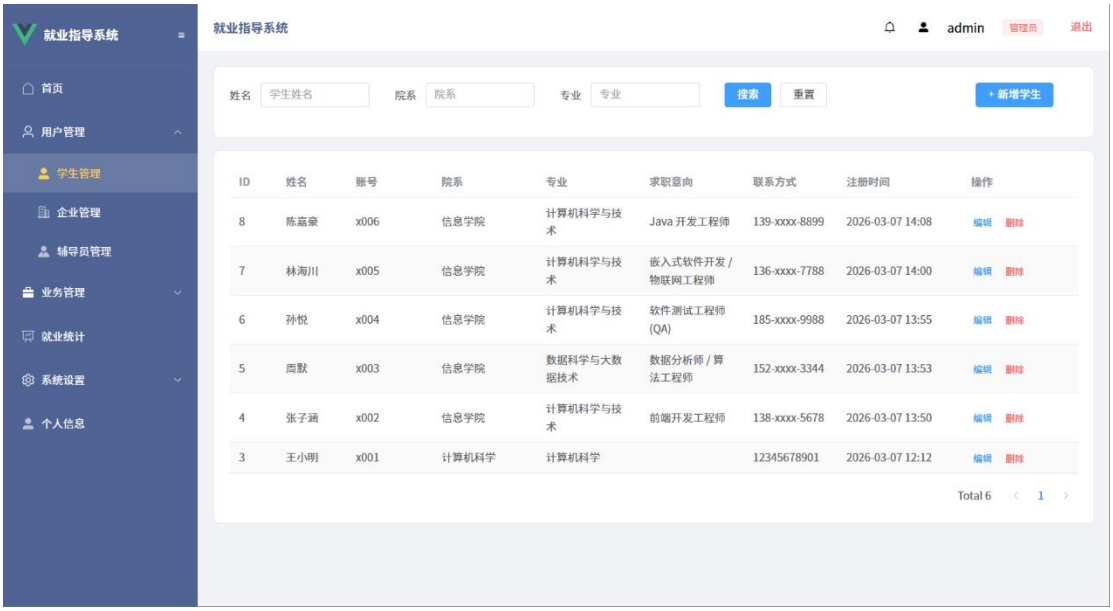


图 5.1.2-2 学生管理页面实现界面

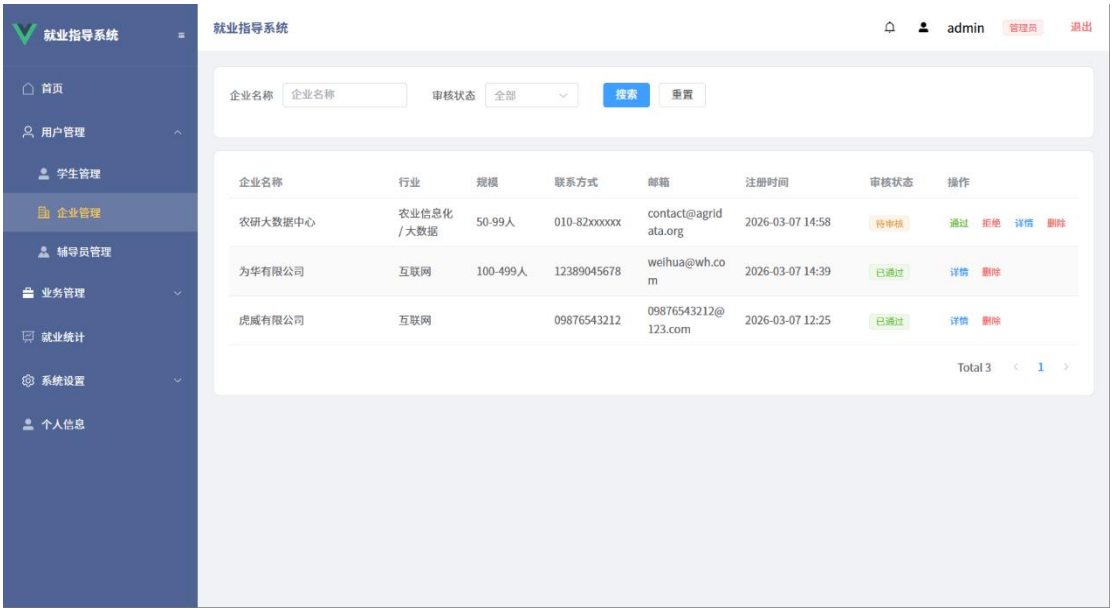


图 5.1.2-3 企业管理页面实现界面

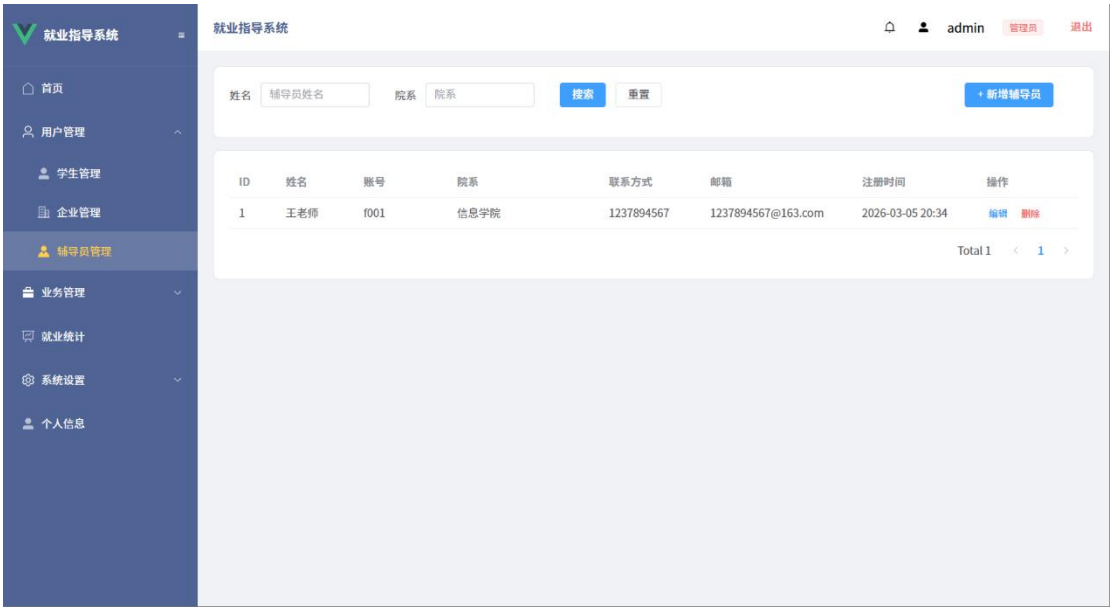


图 5.1.2-4 辅导员管理页面实现界面

（3）业务管理

业务管理目前包含岗位审核功能。页面列出所有企业提交的岗位，管理员可对待审核岗位执行通过或拒绝操作。审核通过的岗位对学生可见，可被正常投递；被拒绝的岗位将从学生端下架，企业需重新提交。实现界面如图 5.1.2-5 所示。

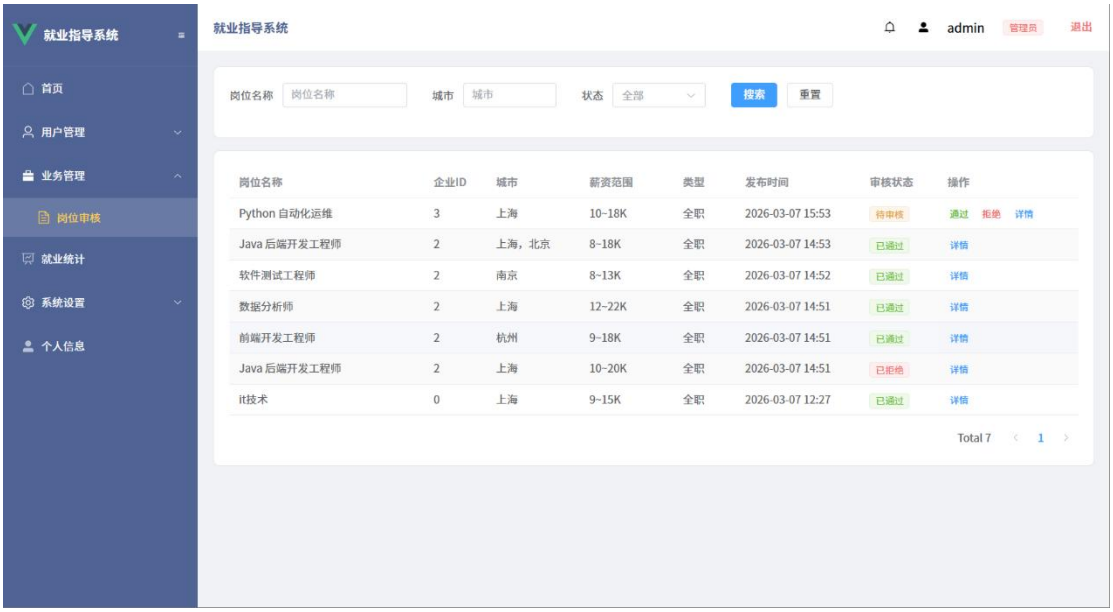


图 5.1.2-5 岗位审核页面实现界面

（4）就业统计

就业统计模块基于 ECharts 实现，包含近 7 天新增学生条形图、全校求职情况环形饼图（已录用/未录用比例）、近 6 个月简历投递趋势图，以及每位学生的投递数、面邀数、录用数明细表，帮助管理员全面掌握学生就业动态。实现界面如图 5.1.2-6 所示。

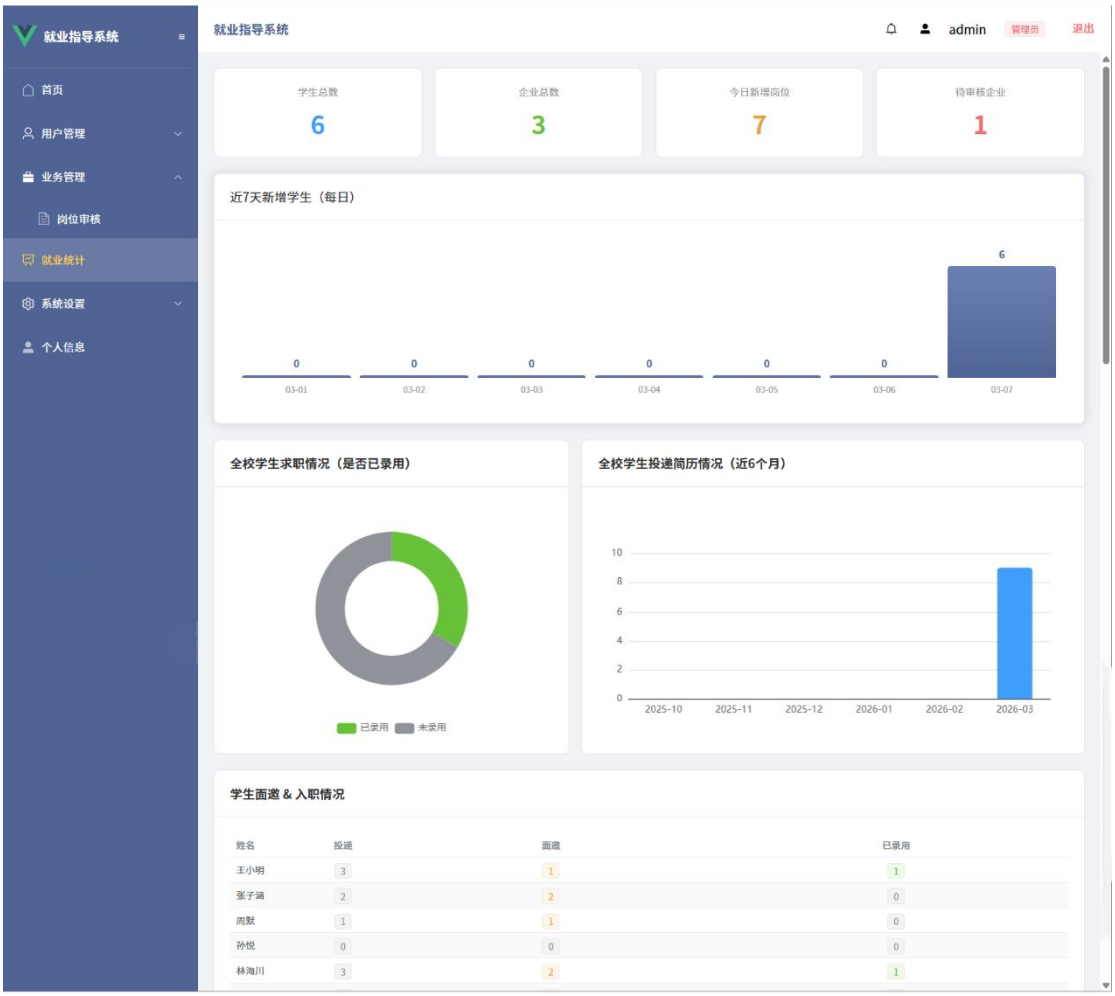


图 5.1.2-6 就业统计页面实现界面

（5）数据字典

数据字典用于维护系统中各类状态字段的枚举值，保证全局数据命名的一致性。管理员可对字典条目进行新增、编辑和删除操作。实现界面如图 5.1.2-7 所示。

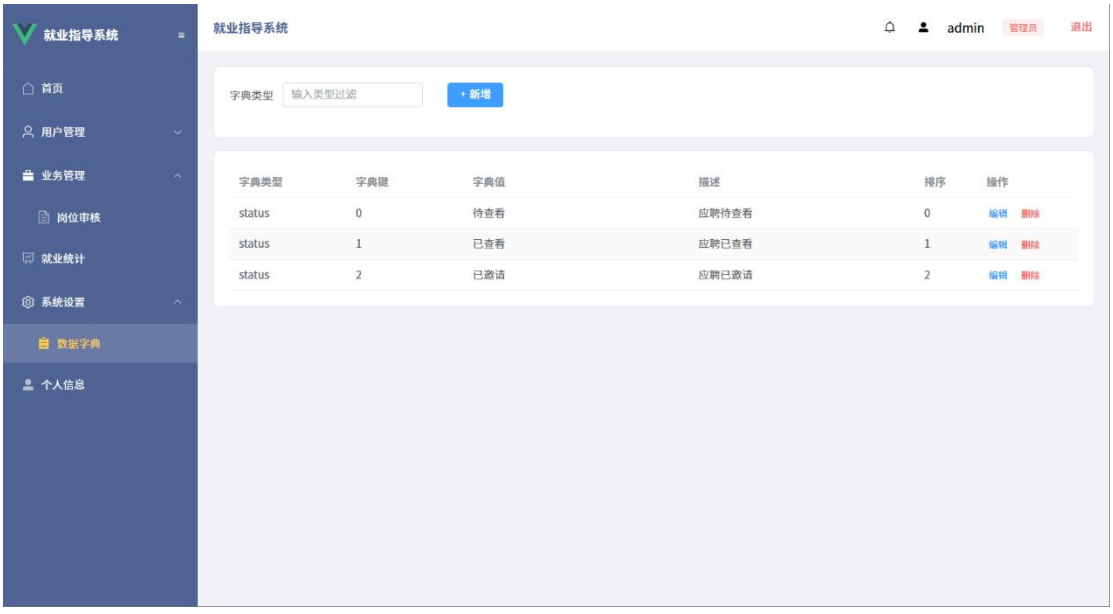


图 5.1.2-7 数据字典页面实现界面

管理员模块实现的部分核心代码如下（首页）：

```
<template>
  <div>
    <el-row :gutter="16" style="margin-bottom: 20px;">
      <el-col :span="6" v-for="card in statCards" :key="card.title">
        <el-card shadow="hover" style="border-radius: 8px;">
          <div style="display: flex; align-items: center; gap: 16px;">
            <el-icon :size="36" :style="{ color: card.color }">
              <component :is="card.icon" />
            </el-icon>
            <div>
              <div style="font-size: 13px; color: #999;">{{ card.title }}</div>
              <div style="font-size: 28px; font-weight: bold; color:
#333;">{{ card.value }}</div>
            </div>
          </div>
        </el-card>
      </el-col>
    </el-row>
    <el-row :gutter="16" style="margin-bottom: 20px;">
      <el-col :span="24">
        <el-card shadow="hover" style="border-radius: 8px;">
          <template #header><span>近 7 天新增学生数</span></template>
          <div style="height: 200px; display: flex; align-items: flex-end; gap: 8px;
padding: 0 8px;">
            <div v-for="item in dailyStudents" :key="item.date"
                 style="flex: 1; display: flex; flex-direction: column; align-items: center;
gap: 4px;">
              <span style="font-size: 12px; color: #333;">{{ item.count }}</span>
              <div :style="{ height: Math.max(item.count * 20, 4) + 'px', background:
'#4f6395', borderRadius: '3px 3px 0 0', width: '100%' }"></div>
              <span style="font-size: 11px; color:
#999;">{{ item.date.slice(5) }}</span>
            </div>
          </div>
```

```
            </el-card>
          </el-col>
        </el-row>
……
<script setup>
import { ref, onMounted } from 'vue'
import { useRouter } from 'vue-router'
import { http } from '@/utils/request'
import { ElMessage } from 'element-plus'

const router = useRouter()
const statCards = ref([
  { title: '总学生数', value: 0, icon: 'UserFilled', color: '#409EFF' },
  { title: '总企业数', value: 0, icon: 'OfficeBuilding', color: '#67C23A' },
  { title: '今日新增岗位', value: 0, icon: 'Briefcase', color: '#E6A23C' },
  { title: '待审核企业', value: 0, icon: 'Warning', color: '#F56C6C' },
])
const dailyStudents = ref([])
const recentCompanies = ref([])

onMounted(async () => {
  try {
    const data = await http.get('/api/statistics/admin/overview')
    statCards.value[0].value = data.totalStudents
    statCards.value[1].value = data.totalCompanies
    statCards.value[2].value = data.todayJobs
    statCards.value[3].value = data.pendingCompanies
    dailyStudents.value = data.dailyStudents || []
    recentCompanies.value = data.recentCompanies || []
  } catch (e) {
    ElMessage.error(e.message)
  }
})

function statusType(s) { return s === 0 ? 'warning' : s === 1 ? 'success' : 'danger' }
```

```
function statusText(s) { return s === 0 ? '待审核' : s === 1 ? '已通过' : '已拒绝' }
function formatTime(t) { return t ? t.replace('T', ' ').slice(0, 16) : '' }
function goAudit() { router.push('/admin/company') }
</script>
```

5.1.3 辅导员管理模块的实现

辅导员登录后可通过左侧导航栏进入首页、学生管理、就业指导和就业统计四大功能模块。

（1）首页

辅导员首页顶部以卡片形式展示三项数据：我的学生总数、本月发布指导帖数和本月发布公开课数。页面下方列出我的学生列表，展示每位学生的用户名、姓名、院系、专业和求职意向，便于辅导员快速了解学生整体情况。实现界面如图 5.1.3-1 所示。



图 5.1.3-1 辅导员首页实现界面

（2）学生管理

学生管理页面展示该辅导员名下所有学生的详细信息，包括账号、姓名、院系、专业、求职意向和联系方式，支持按姓名检索。实现界面如图 5.1.3-2 所示。

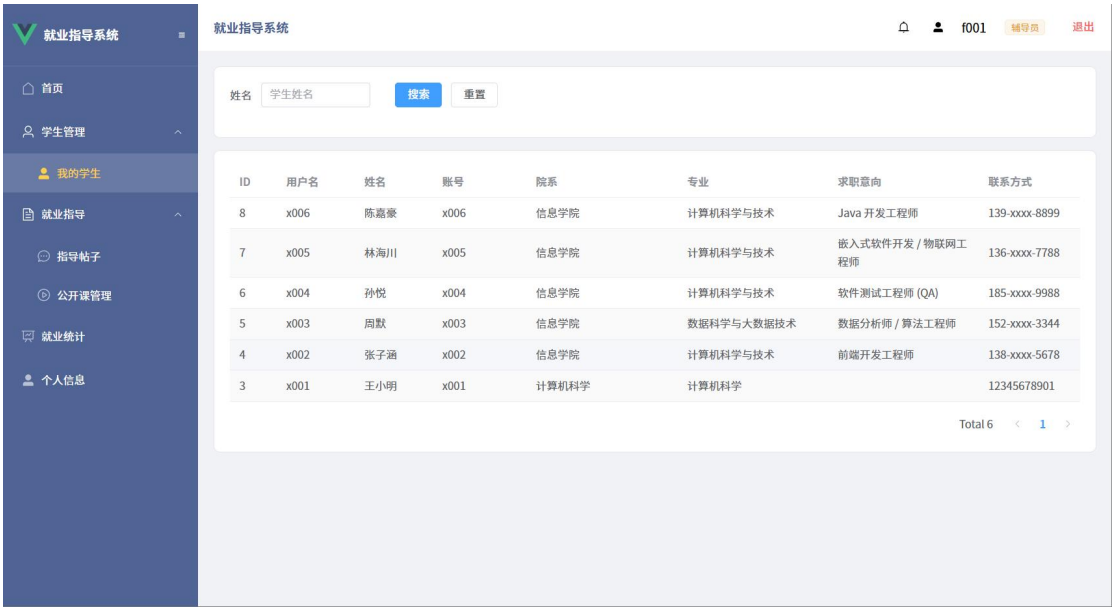


图 5.1.3-2 我的学生页面实现界面

（3）就业指导

就业指导模块分为指导帖子和公开课管理两个子页面。

指导帖子页面以卡片形式展示辅导员发布的所有帖子，每张卡片显示标题、发布时间和内容摘要，支持新增、编辑和删除操作。实现界面如图 5.1.3-3 所示。

公开课管理页面以列表形式展示已发布的公开课，包含课程名称、开课时间、报名人数、人数上限和当前状态，支持新增、编辑、删除操作，还可点击"名单"查看已报名学生列表。实现界面如图 5.1.3-4 所示。



图 5.1.3-3 指导帖子页面实现界面



图 5.1.3-4 公开课管理页面实现界面

辅导员模块实现的部分核心代码如下（学生管理页面）：

```
<template>
  <div>
    <el-card shadow="never" style="margin-bottom: 16px; border-radius: 8px;">
      <el-form :model="query" inline>
        <el-form-item label="姓名">
          <el-input v-model="query.name" placeholder="学生姓名" clearable
style="width: 140px;" />
        </el-form-item>
        <el-form-item>
          <el-button type="primary" @click="loadList">搜索</el-button>
          <el-button @click="resetQuery">重置</el-button>
        </el-form-item>
      </el-form>
    </el-card>
    <el-card shadow="never" style="border-radius: 8px;">
      <el-table :data="list" v-loading="loading" stripe>
        <el-table-column prop="id" label="ID" width="60" />
        <el-table-column prop="userName" label="用户名" width="100" />
        <el-table-column prop="realName" label="姓名" width="100" />
        <el-table-column prop="userName" label="账号" width="120" />
        <el-table-column prop="department" label="院系" />
```

```
            <el-table-column prop="major" label="专业" />
            <el-table-column prop="jobIntention" label="求职意向" />
            <el-table-column prop="phone" label="联系方式" width="130" />
          </el-table>
          <el-pagination style="margin-top: 12px; justify-content: flex-end;"
            layout="total, prev, pager, next"
            :total="total" :page-size="pageSize" v-model:current-page="page"
            @current-change="loadList" />
        </el-card>
      </div>
    </template>
    <script setup>
    import { ref, reactive, onMounted } from 'vue'
    import { http } from '@/utils/request'
    import { ElMessage } from 'element-plus'
    const list = ref([])
    const total = ref(0)
    const page = ref(1)
    const pageSize = 10
    const loading = ref(false)
    const query = reactive({ name: '' })
    const counselorId = localStorage.getItem('userId') || ''
    onMounted(loadList)
    async function loadList() {
      loading.value = true
      try {
        const res = await http.get('/api/student/list', {
          page: page.value, size: pageSize, name: query.name, counselorId
        })
        list.value = res.records || []
        total.value = res.total || 0
      } catch (e) { ElMessage.error(e.message) }
      finally { loading.value = false }
    }
```

```
function resetQuery() {
  query.name = ''
  page.value = 1
  loadList()
}
</script>
```

5.1.4 学生管理模块的实现

学生模块是系统功能最丰富的部分，登录后可通过左侧导航栏进入首页、我的简历、岗位推荐、岗位搜索、应聘记录、就业指导和交流中心。

（1）首页

学生登录后进入个性化首页，顶部横幅展示学生姓名并附加激励语，页面中部提供完善简历、查看推荐岗位、我的投递、就业指导四个快捷入口，下方以卡片形式展示系统为该学生推荐的岗位列表，点击"查看详情"可直接跳转投递。实现界面如图 5.1.4-1 所示。

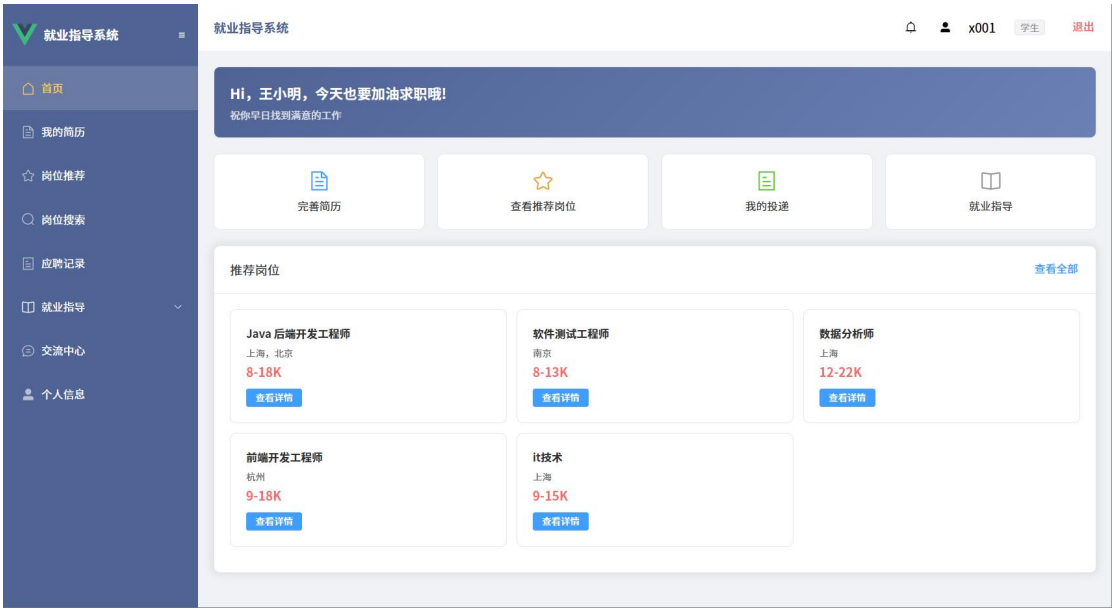


图 5.1.4-1 学生首页实现界面

（2）我的简历

简历页面包含基本信息、技能标签、教育背景、工作/实习经历和自我评价五个部分，学生可随时完善并保存。其中技能标签以标签形式展示，支持自由添加和删除，是系统进行岗位匹配推荐的重要依据。实现界面如图 5.1.4-2 所示。



图 5.1.4-2 我的简历页面实现界面

（3）岗位搜索与推荐

岗位搜索页面支持按关键词和城市检索，点击岗位卡片可弹出岗位详情弹窗，展示岗位名称、城市、薪资范围、学历要求、技能要求及岗位描述，确认后可直接点击"立即投递"提交简历，已投递的岗位不可重复投递。岗位推荐页面的入口和界面交互与岗位搜索一致，区别在于展示的岗位列表由系统根据学生简历标签智能匹配生成。实现界面如图 5.1.4-3 所示。

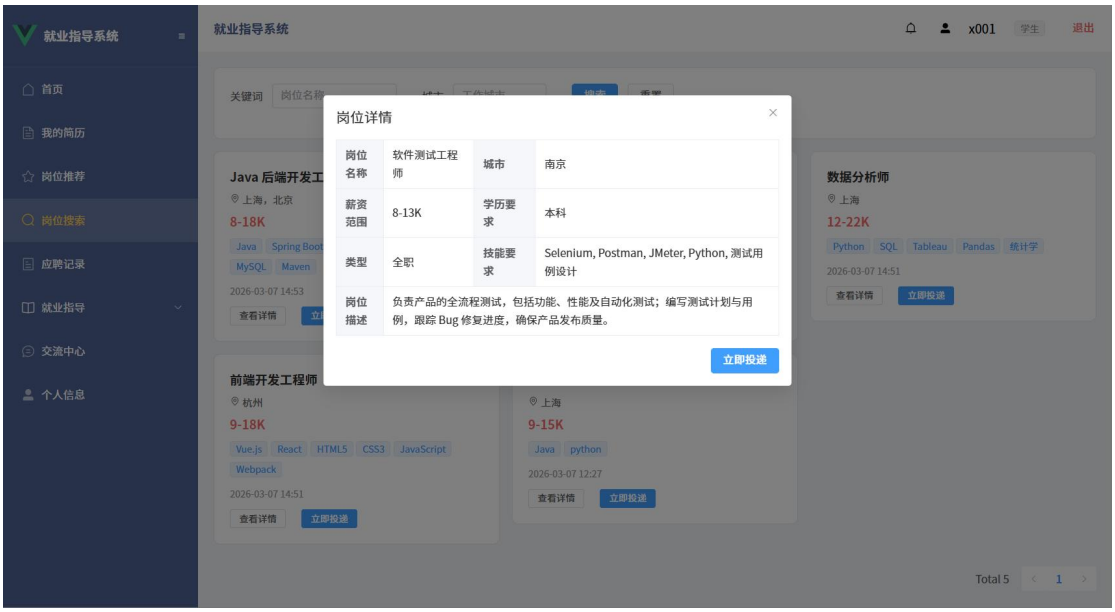


图 5.1.4-3 岗位搜索与详情页面实现界面

（4）就业指导

就业指导模块分为指导帖子和公开课两个子页面。指导帖子页面展示辅导员发布的就

业指导内容供学生查阅；公开课页面列出所有可报名的课程，显示开课时间和剩余名额，学生可直接在线报名或取消报名。实现界面如图 5.1.4-4 所示。



图 5.1.4-4 公开课页面实现界面

（5）交流中心

交流中心采用左右分栏布局，左侧为帖子列表，右侧展示帖子正文与回复内容。学生可自由发帖，分享求职经历或寻求帮助，其他同学可在帖子下方回复，形成互助交流的社区氛围。实现界面如图 5.1.4-5 所示。



图 5.1.4-5 交流中心页面实现界面

学生模块实现的部分核心代码如下（我的简历页面）：

```
<el-divider content-position="left">技能标签</el-divider>
<div style="display: flex; flex-wrap: wrap; gap: 8px; margin-bottom: 12px;">
```

```
            <el-tag v-for="skill in skills" :key="skill" closable
@close="removeSkill(skill)">{{ skill }}</el-tag>
            <el-input v-if="addingSkill" v-model="newSkill" size="small" style="width:
100px;"
                      @keyup.enter="confirmSkill" @blur="confirmSkill" ref="skillInput" />
            <el-button v-else size="small" @click="startAddSkill">+ 添加技能</el-button>
          </div>
          <el-divider content-position="left">教育背景</el-divider>
          <el-input v-model="form.eduHistory" type="textarea" :rows="3"
                    placeholder="请描述您的教育经历" />
          <el-divider content-position="left">工作/实习经历</el-divider>
          <el-input v-model="form.workHistory" type="textarea" :rows="4"
                    placeholder="请描述您的工作或实习经历" />
          <el-divider content-position="left">自我评价</el-divider>
          <el-input v-model="form.selfEval" type="textarea" :rows="4"
                    placeholder="请简要介绍自己的优势和特点..." />
        </el-form>
      </el-card>
    </div>
</template>
      <script setup>
      import { ref, reactive, nextTick, onMounted } from 'vue'
      import { http } from '@/utils/request'
      import { ElMessage } from 'element-plus'
      const saving = ref(false)
      const addingSkill = ref(false)
      const newSkill = ref('')
      const skillInput = ref(null)
      const skills = ref([])
      const studentId = parseInt(localStorage.getItem('userId') || '0')
      const form = reactive({
        studentId: studentId,
        jobIntention: '',
        cityIntention: '',
        education: '',
```

```
  skills: '',
  eduHistory: '',
  workHistory: '',
  selfEval: '',
})
onMounted(async () => {
  if (!studentId) return
  try {
    const data = await http.get('/api/resume/my', { studentId })
    if (data) {
      Object.assign(form, data)
      skills.value = (data.skills || '').split(',').filter(Boolean)
    }
  } catch (e) { /* 无简历则新建 */ }
})
function startAddSkill() {
  addingSkill.value = true
  nextTick(() => skillInput.value?.focus())
}
function confirmSkill() {
  if (newSkill.value.trim()) skills.value.push(newSkill.value.trim())
  newSkill.value = ''
  addingSkill.value = false
}
function removeSkill(skill) {
  skills.value = skills.value.filter(s => s !== skill)
}
async function save() {
  form.skills = skills.value.join(',')
  saving.value = true
  try {
    await http.post('/api/resume/save', form)
    ElMessage.success('简历保存成功')
  } catch (e) { ElMessage.error(e.message) }
  finally { saving.value = false }
```

```
}
</script>
```

5.1.5 企业管理模块的实现

企业用户审核通过后即可登录系统，通过左侧导航栏进入首页、岗位管理、求职者推荐和应聘管理四大功能模块。

（1）首页

企业首页顶部展示三项核心数据：发布岗位数、收到简历数和待处理面试邀请数。下方列出最新投递记录，包含学生姓名、应聘岗位、投递时间和当前状态，企业可点击"查看"弹出投递详细流程弹窗，以时间轴形式呈现从提交申请、企业查看、收到面试邀请、学生确认到最终录用的完整流程节点。实现界面如图 5.1.5-1、图 5.1.5-2 所示。

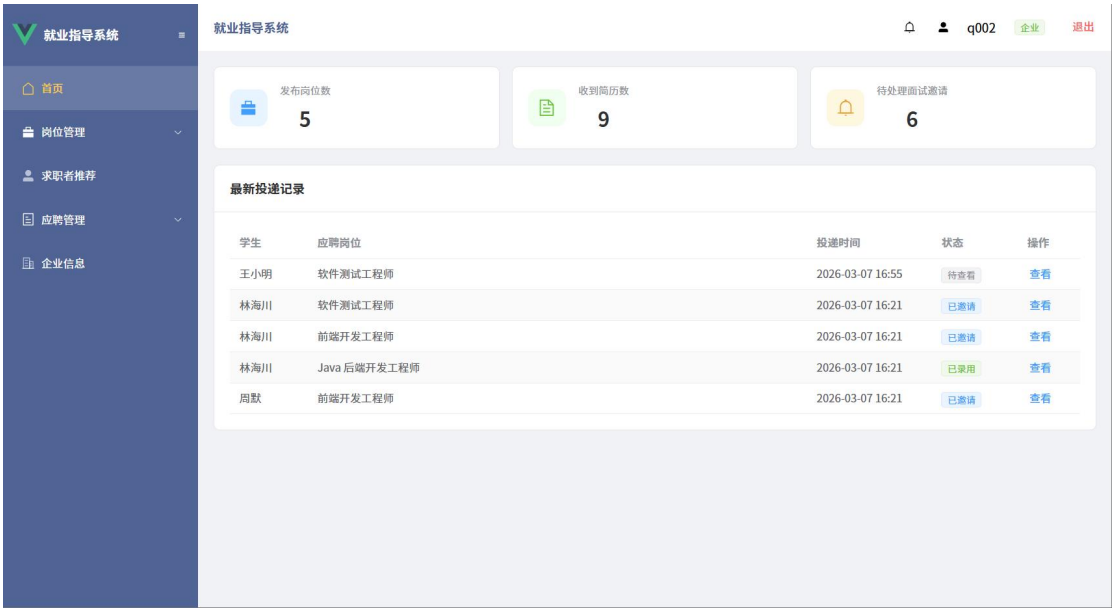


图 5.1.5-1 企业首页实现界面

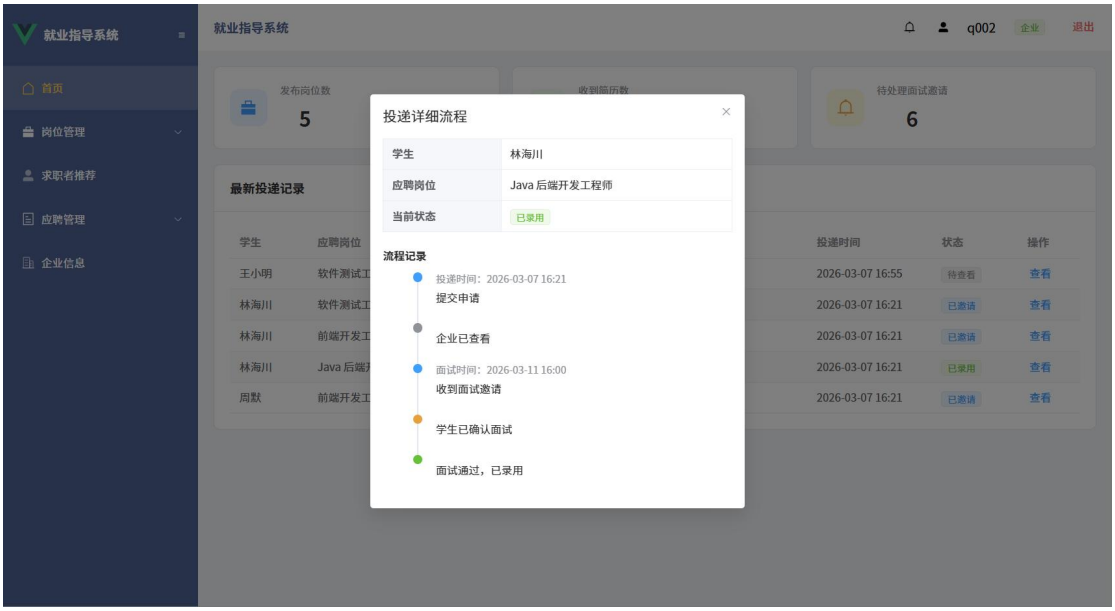


图 5.1.5-2 投递详细流程弹窗实现界面

（2）岗位管理

岗位管理页面列出企业已发布的所有岗位，展示岗位名称、城市、薪资范围、类型、学历要求及当前审核状态，支持编辑和删除操作。点击"发布岗位"弹出表单，填写岗位名称、工作城市、薪资范围、学历要求、岗位类型、技能要求和岗位描述后提交，岗位进入待审核状态，审核通过后才对学生可见。实现界面如图 5.1.5-3、图 5.1.5-4 所示。



图 5.1.5-3 我发布的岗位页面实现界面

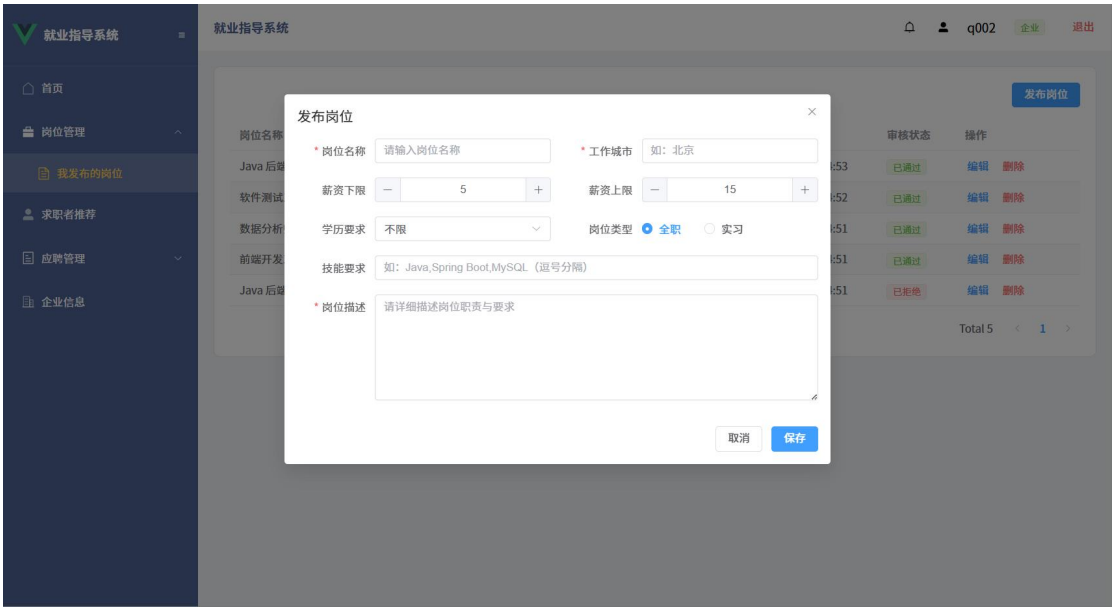


图 5.1.5-4 发布岗位表单实现界面

（3）求职者推荐

求职者推荐页面根据企业已发布的岗位需求，自动为企业推荐匹配度高的求职者。每张求职者卡片展示专业、院系、求职意向和技能标签，企业可点击"查看简历"查阅详细信息，也可直接点击"发送面试邀请"进入招聘流程。实现界面如图 5.1.5-5 所示。

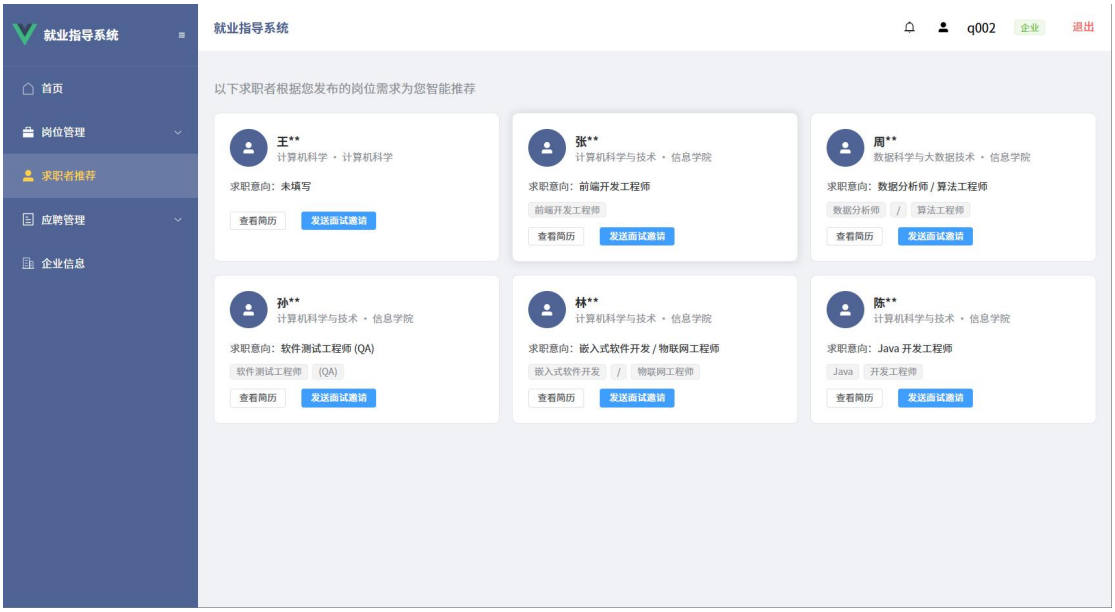


图 5.1.5-5 求职者推荐页面实现界面

（4）应聘管理

应聘管理分为投递管理和面试管理两个子页面，完整的应聘流程如图 5.1.5-6 所示。

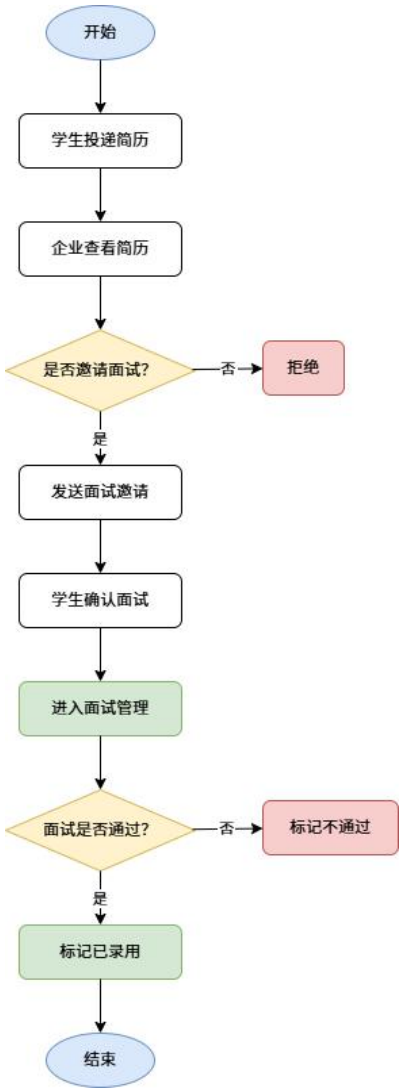


图 5.1.5-6 应聘管理操作流程图

在投递管理页面，企业的岗位以横向标签页排列，点击对应岗位可查看该岗位下所有投递学生的状态。企业可查看学生简历，并对待处理的投递执行邀请面试或拒绝操作；已邀请的学生需在线确认后，该条记录才会转移至面试管理。面试管理页面汇总所有进入面试阶段的学生，企业可在此标记面试通过或不通过，录用的学生以高亮状态显示，不通过的学生信息将变灰。实现界面如图 5.1.5-7、图 5.1.5-8 所示。



图 5.1.5-7 投递管理页面实现界面



图 5.1.5-8 面试管理页面实现界面

企业模块实现的部分核心代码如下（应聘管理页面）：

```
<template>
  <div>
    <el-tabs v-model="mainTab" @tab-change="onMainTabChange">
      <!--投递管理 -->
      <el-tab-pane label="投递管理" name="apply">
        <el-card shadow="never" style="border-radius:8px; margin-top:8px;"
v-loading="loading">
          <el-tabs v-if="jobs.length > 0" v-model="activeJob"
```

```
@tab-change="loadApps">
                    <el-tab-pane v-for="job in
jobs" :key="job.id" :label="job.title" :name="String(job.id)">
                      <el-table :data="appList" v-loading="appLoading" stripe
style="margin-top:12px;"
                        :row-style="({ row }) => row.status === 4 ? { color:'#c0c4cc' } : {}">
                        <el-table-column prop="studentId" label="学生 ID" width="80" />
                        <el-table-column prop="applyTime" label="投递时间" width="160">
                          <template #default="{ row }">{{ fmt(row.applyTime) }}</template>
                        </el-table-column>
                        <el-table-column label="状态" width="110">
                          <template #default="{ row }">
                            <el-tag :type="statusType(row.status)"
size="small">{{ statusText(row.status) }}</el-tag>
                          </template>
                        </el-table-column>
                        <el-table-column prop="interviewTime" label="面试时间"
width="160">
                          <template
#default="{ row }">{{ fmt(row.interviewTime) }}</template>
                        </el-table-column>
                        <el-table-column label="操作" fixed="right" width="230">
                          <template #default="{ row }">
                            <el-button type="primary" link
@click="viewResume(row.studentId)">查看简历</el-button>
                            <template v-if="row.status <= 1">
                              <el-button type="warning" link @click="openInvite(row)">邀
请面试</el-button>
                              <el-button type="danger" link @click="doUpdateStatus(row.id,
4, false)">拒绝</el-button>
                            </template>
                            <span v-else-if="row.status === 2"
                              style="font-size:12px; color:#909399; margin-left:4px;">待学
生确认</span>
                          </template>
```

```
                        </el-table-column>
                    </el-table>
                    <el-pagination style="margin-top:12px; justify-content:flex-end;"
                        layout="total, prev, pager, next"
                        :total="appTotal" :page-size="10" v-model:current-page="appPage"
                        @current-change="loadApps" />
                </el-tab-pane>
            </el-tabs>
            <el-empty v-else description="暂无发布的岗位" />
        </el-card>
    </el-tab-pane>
    <!-- 面试管理 -->
    <el-tab-pane label="面试管理" name="interview">
        <el-card shadow="never" style="border-radius:8px; margin-top:8px;"
v-loading="interviewLoading">
            <el-table :data="interviewList" stripe
                :row-style="({ row }) => row.status === 6 ? { color:'#c0c4cc' } : {}">
                <el-table-column prop="studentName" label="学生" width="100" />
                <el-table-column prop="jobTitle" label="应聘岗位" min-width="150" />
                <el-table-column prop="interviewTime" label="面试时间" width="160">
                    <template #default="{ row }">{{ fmt(row.interviewTime) }}</template>
                </el-table-column>
                <el-table-column label="状态" width="120">
                    <template #default="{ row }">
                        <el-tag :type="statusType(row.status)"
size="small">{{ statusText(row.status) }}</el-tag>
                    </template>
                </el-table-column>
                <el-table-column label="操作" fixed="right" width="220">
                    <template #default="{ row }">
                        <el-button type="primary" link @click="viewResume(row.studentId)">
查看简历</el-button>
                        <template v-if="row.status === 5">
                            <el-button type="success" link @click="doUpdateStatus(row.id, 3,
true)">通过</el-button>
```

```
                                <el-button type="danger" link @click="doUpdateStatus(row.id, 6,
true)">不通过</el-button>
                            </template>
                        </template>
                    </el-table-column>
                </el-table>
                <el-empty v-if="!interviewLoading && !interviewList.length" description="暂
无面试记录" />
            </el-card>
        </el-tab-pane>
    </el-tabs>
    <!-- 简历弹窗 -->
    <el-dialog v-model="resumeVisible" title="查看简历" width="600px">
        <div v-if="resume">
            <el-descriptions :column="2" border>
                <el-descriptions-item label="求职意向
">{{ resume.jobIntention }}</el-descriptions-item>
                <el-descriptions-item label="意向城市
">{{ resume.cityIntention }}</el-descriptions-item>
                <el-descriptions-item label="技能">{{ resume.skills }}</el-descriptions-item>
                <el-descriptions-item label="联系方式
">{{ resume.phone }}</el-descriptions-item>
            </el-descriptions>
            <div style="margin-top:12px;">
                <div style="font-weight:600; margin-bottom:4px;">自我评价</div>
                <div style="color:#606266;">{{ resume.selfEval || '暂无' }}</div>
            </div>
        </div>
        <el-empty v-else description="该学生暂无简历" />
    </el-dialog>
    <!-- 邀请面试弹窗 -->
    <el-dialog v-model="inviteVisible" title="邀请面试" width="400px">
        <el-form label-width="90px">
            <el-form-item label="面试时间">
                <el-date-picker v-model="inviteTime" type="datetime" placeholder="选择面
```

```
试时间" style="width:100%;" />
                </el-form-item>
            </el-form>
            <template #footer>
                <el-button @click="inviteVisible = false">取消</el-button>
                <el-button type="primary" :loading="saving" @click="confirmInvite">确认邀请
</el-button>
            </template>
        </el-dialog>
    </div>
</template>
```

5.1.6 AI 推荐模块的实现

AI 推荐模块主要在企业用户页面与学生用户页面。对于学生用户，AI 推荐会根据其简历信息，推荐与自身技能相匹配的岗位；对于企业用户，AI 推荐则根据企业发布的岗位需求，为其推荐符合条件的求职者。进入该模块后，系统会自动完成初步分析并推送推荐结果，每条推荐附有匹配理由，支持直接投递或发送面试邀请。此外，用户也可以继续追问，例如简历修改建议、筛选条件补充等，AI 都会结合上下文耐心解答。学生端与企业端 AI 推荐实现界面分别如图 5.1.6-1、图 5.1.6-2 所示。



图 5.1.6-1 学生端 AI 岗位推荐实现界面



图 5.1.6-2 企业端 AI 求职者推荐实现界面

5.2 系统测试

系统测试是软件开发过程中不可缺少的环节，目的是对软件质量进行系统确认，验证系统是否完成了需求分析阶段所提出的各项要求，确认实现方式的正确性。测试过程也能为开发人员提供有效的反馈信息，为风险评估做好准备。软件测试的范围不仅是软件产品本身，还包括整个开发过程，一个软件在上线后频繁暴露问题，往往说明开发过程本身存在缺陷。因此，软件测试是保障高质量软件开发过程的重要手段[41][42]。

5.2.1 系统测试环境

因本系统采用的是 B/S 结构模式，对 web 客户端的配置要求不是特别高，通常情况下一般的浏览器即可满足要求。本系统用的是 Google 浏览器作为测试环境。

硬件测试环境配置如下表 5.2.1-1 所示：

表 5.2.1-1 硬件测试环境

配置项	说明
设备型号	微星 MSI 笔记本
CPU	12 核处理器
内存	16GB
硬盘	1TB

软件测试环境配置如下表 5.2.1-2 所示：

表 5.2.1-2 软件测试环境

配置项	说明
操作系统	Windows 11
浏览器	Google Chrome
前端运行环境	Node.js
后端运行环境	Spring Boot 3.3.0 / JDK 17
数据库管理工具	Navicat Premium 17

5.2.2 系统测试用例

目前较为主流的测试方法有黑盒测试与白盒测试两类，本系统结合两种方法对各功能模块进行测试验证。

黑盒测试又称功能测试，测试人员不关注系统内部实现逻辑，仅通过输入数据并观察输出结果来判断系统功能是否符合预期，适用于对系统业务功能的验证[43]。白盒测试又称结构测试，测试人员需要了解系统内部代码结构与执行路径，通过设计覆盖各分支逻辑的测试用例来发现潜在的代码缺陷[44]。本系统主要采用黑盒测试方法，针对各功能模块的业务流程设计测试用例，验证系统输出是否符合预期。

表 5.2.2-1 系统登录模块测试用例

测试目标	测试步骤	预期结果	测试结果
用户登录验证	输入正确账号和密码 q001 q001123	跳转至对应角色首页	通过
	不输入账号，直接点击登录	提示"请输入用户名"	通过
	输入正确账号，密码输错	提示"账号或密码错误"	通过
	连续输入错误密码 5 次	账号锁定，提示点击忘记密码重置	通过
	使用学生账号选择企业角色登录	提示角色不匹配，登录失败	通过

表 5.2.2-2 企业用户权限模块测试用例

测试目标	测试步骤	预期结果	测试结果
用户	创建新用户，注册页面选择学生，输入 x001 密码 x001123 姓名王小明	注册成功，跳转登录页面	通过

权限测试	查询学生"王小明"	查询成功	通过
	创建新用户，注册页面选择学生，输入 x，密码为空	注册失败，提示密码不能为空	通过
	新注册企业登录后尝试使用各功能	顶部提示审核中，功能受限，仅可查看首页	通过
	企业发布岗位后，学生用户搜索该岗位	审核通过前学生用户不可见	通过
	管理员审核通过后企业重新登录或刷新	所有功能正常使用	通过
	学生用户投递推荐岗位	投递成功	通过
	管理员删除某企业账号，该账号尝试登录	提示账号不存在，登录失败	通过

每个模块的测试大同小异不再重复例举，现将部分模块具体测试过程及界面截图展现出来。

5.2.3 系统测试结果

（1）安全性能测试结果

针对登录模块进行安全性测试，当用户连续输入错误密码达 5 次后，系统会自动锁定该账号并弹出提示，要求用户点击"忘记密码"通过邮箱验证重置密码后方可重新登录，有效防止暴力破解。测试结果界面如图 5.2.3-1 所示。



图 5.2.3-1 账号锁定提示界面

（2）用户权限测试结果

针对企业用户进行权限测试，新注册的企业账号在未通过管理员审核前，登录后系统

顶部会展示审核提示横幅，所有功能入口均处于受限状态，仅首页可以正常访问。企业发布的岗位同样需要经过管理员审核，审核通过前不会出现在学生端的岗位列表中，保障平台内容的真实性与可靠性。测试结果界面如图 5.2.3-2、图 5.2.3-3 所示。

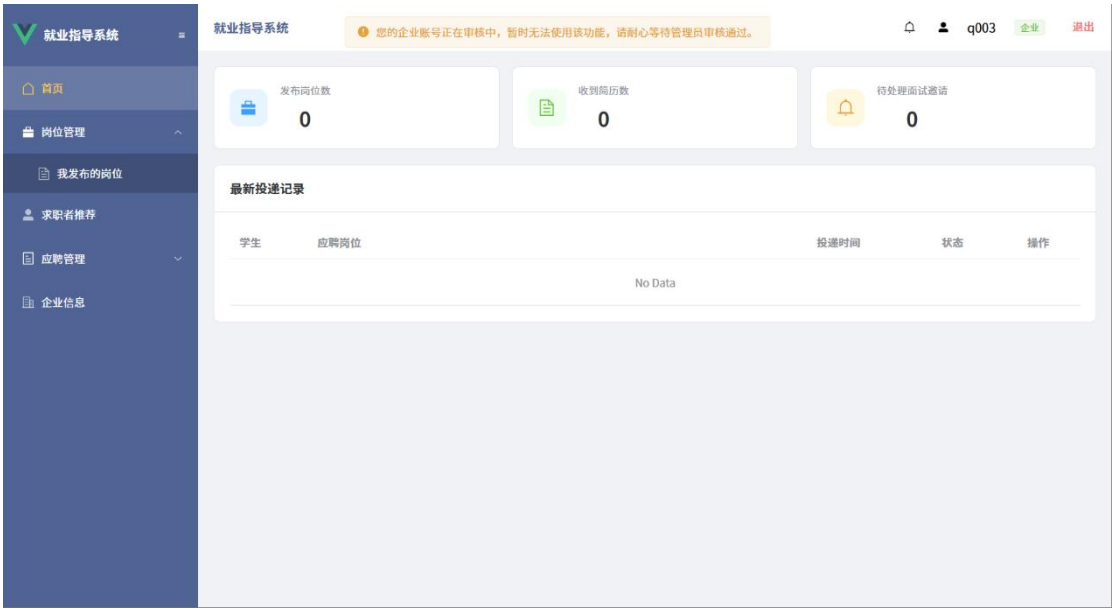


图 5.2.3-2 企业账号待审核状态界面



图 5.2.3-3 企业岗位待审核状态界面

5.3 本章小结

本章对就业指导系统的实现与测试进行了介绍。实现部分按登录注册、管理员、辅导员、学生、企业、AI 推荐六大模块依次展示了各模块的功能与实现界面。测试部分对系统测试环境进行了说明，针对登录安全性与用户权限控制设计了测试用例并进行了验证，测试结果均符合预期。

第六章 结论与展望

本文以应届生求职困境为出发点，针对市面上现有招聘平台普遍从企业视角出发、缺乏真正服务求职者的工具这一现状，设计并开发了一套面向应届生的就业指导系统，以上海海洋大学作为初期测试用例。

系统采用 Vue3 + Element Plus 作为前端框架，Spring Boot 3.3.0 + MyBatis-Plus 作为后端框架，MySQL 作为数据库，整体采用 B/S 架构，支持学生、企业、辅导员、管理员四类用户角色。围绕"学生求职"这条主线，系统涵盖简历管理、岗位推荐、岗位搜索、应聘记录追踪、就业指导、交流中心等核心功能，企业页面支持岗位发布与招聘流程管理，辅导员页面支持就业指导内容发布与学生数据统计，管理员页面负责系统整体的运维与审核工作。此外，系统还接入了字节跳动旗下 Doubao-Seed-2.0-mini 大模型，为学生提供个性化岗位推荐与简历优化建议，为企业提供智能求职者推荐，实现了 AI 驱动的双向精准匹配。通过系统测试，各功能模块均运行正常，权限控制与安全机制符合预期，系统整体稳定可靠，基本满足了应届生求职场景下的核心需求。

由于开发周期有限，系统目前仍处于起步阶段，部分功能还不够完善。在推荐算法上，后续可引入更丰富的用户行为数据，进一步提升推荐精准度；在用户群体上，可逐步扩展至往届生、实习生等更广泛的求职群体；此外，后续也可考虑开发移动端版本或对接微信公众号，让学生随时随地查看推荐岗位、追踪应聘进展，进一步提升系统的使用便捷性。

参考文献

[1] Xu Xiaoxia. Development of a Socio-Technical Deep Neural Network Framework for Personalized Career Guidance and Entrepreneurship Training: A Systems Approach[J].International Journal of Information Technologies and Systems Approach (IJITSA),2025,18(1):

[2] Hajar Majjate,Youssra Bellarhmouch,Adil Jeghal,Ali Yahyaouy,Hamid Tairi,Khalid Alaoui Zidani. AI-Powered Academic Guidance and Counseling System Based on Student Profile and Interests[J].Applied System Innovation,2023,7(1).

[3] Siyue Cha,Hong Li. Design and Implementation of College Students' Employment Guidance System Based on Knowledge Graph[J].The Frontiers of Society, Science and Technology,2023,5(15).

[4] Juan Sánchez, Fabio Sánchez, Helbert Espitia. Artificial neuro-fuzzy system for the formulation of guidelines in the process of formulating public policies to promote agricultural employment in Colombia[J] Environmental Development, 2025, 101290.

[5] Huan Yu, Ru Zhang, Cheonshik Kim. Intelligent analysis system of college students' employment and entrepreneurship situation: Big data and artificial intelligence-driven approach [J]. Computers and Electrical Engineering,2023,108823.

[6] Yu Wang. Design and implementation of student job matching system based on personalized recommendation algorithm [J]. Sustainable Agriculture and Food Security,2025,200302.

[7] Juan Chen. Construction of career planning and employment guidance system for college students[J].Applied Mathematics and Nonlinear Sciences,2024,9(1).

[8] Peisheng Wu. Construction Research on the Employment Guidance System for Vocational College Students[J].Advances in Vocational and Technical Education,2023,5(10).

[9] Jun Zheng, Jundong Tang, Jiansheng Gao. Construction of Graduate Employment Service System Based on Public Information Platform [J]. Applied Mechanics and Materials,2012,102-104:3007-3010.

[10] 王一栋,杨晨,娄旭. 职业生涯规划教育破解大学生就业难题的实践路径探究[J].河南教育(高教),2025,(11):61-62.

[11] 黄克强,柳志余,谢金香,齐桂敏. 新时代高职院校毕业生高质量就业路径探析[J].辽宁农业职业技术学院学报,2025,27(06):63-67.

[12] 李青. 以“一站式”学生社区 推进大学生就业工作[J].中国就业,2025,(11):56-57.

[13] 朱红维. 新形势下高校毕业生就业难问题研究及应对措施[J].新西部,2025,(10):152-155.

[14] 郑露,吴迪. 新时代高职院校毕业生“慢就业”现象的根源与破解路径探索与实践——以许昌市某高校为例[J].才智,2025,(33):121-124.

[15] 王洁,杨文金. 大学生“慢就业”现象的系统分析与应对[J].山西青年职业学院学报,2023,36(01):26-29.

[16] 李佳莹,殷程程,邱越,贾晨晨,朱博闻,刘意麟. “00 后”毕业生内卷与就业焦虑对策研究[J].产业与科技论坛,2025,24(21):78-80.

[17] 周洪兴. 基于 AI 智能体应用的大学生就业指导精准化服务探索[J].河南教育(高教),2025,(11):65-66.

[18] 倪前龙. 基于内容过滤技术的大学生就业指导的研究[J].电脑知识与技术,2025,21(28):48-50+54.

[19] 黄金波,蒲章雨,王杰,吴金忠,王晓伟. 一种高校“T”型人才简历管理推荐系统[J].中国科技信息,2021,(19):82-87+13.

[20] 王宁. 基于 Bootstrap 框架的高校学生就业信息管理系统设计[J].自动化技术与应用,2021,40(04):175-178.

[21] 李燕燕. 基于“互联网+”平台的高校创业就业指导平台设计[J].自动化技术与应用,2020,39(09):143-147.

[22] 白岚. 基于 HTML5 高职院校校友互动就业指导系统的开发与应用[J].电脑编程技巧与维护,2020,(05):103-105+117.

[23] 胡思文. 基于BlackBoard平台的大学生职业生涯与就业指导网络课程教学系统设计[J].数码世界,2017,(09):27.

[24] 戴琼洁,范绍勇. 数字化校园建设中就业指导中心信息系统设计[J].时代金融,2017,(14):280-281.

[25] 窦凯. 对高职院校就业指导工作的若干思考[J].智库时代,2017,(13):251-252.

[26] 廖列营. 基于系统论的大学生就业指导核心建构[J].广东技术师范学院学报,2016,37(08):80-83.

[27] 薛鑫. 高职大学生就业指导与职业生涯规划系统设计研究[J].办公自动化,2023,28(14):28-30.

[28] 梁诗宇.基于 CRM 的高校就业指导系统的设计与实现[D]. 导师：赵明;陆宜梅. 沈阳建筑大学, 2017.

[29] 韩万江. 软件工程案例教程：软件项目开发实践[M]. 北京: 机械工业出版社, 2011.

[30] 黄兴荣. 基于 B/S 架构模式的三层结构设计与实现[J]. 电脑知识与技术, 2015, 11(32):

52-53.

[31] 邱璐璐, 陈俊仁. 基于SpringBoot和Vue框架的高校代领系统设计与实现[J]. 信息技术与信息化, 2021,(04): 25-27.

[32] 尤雨溪等. Vue.js 官方文档[EB/OL]. https://cn.vuejs.org.

[33] 霍春阳. Vue.js 设计与实现[M]. 北京: 人民邮电出版社, 2022.

[34] Craig Walls. Spring Boot 实战[M]. 北京: 人民邮电出版社, 2016.

[35] [美] Rod Johnson 等. Spring 实战（第 5 版）[M]. 北京: 人民邮电出版社, 2020.

[36] MyBatis 官方文档[EB/OL]. https://mybatis.org.

[37] Paul DuBois. MySQL 技术内幕[M]. 北京: 人民邮电出版社, 2012.

[38] 姜承尧. MySQL 技术内幕：InnoDB 存储引擎（第 2 版）[M]. 北京: 机械工业出版社, 2013.

[39] 王珊, 萨师煊. 数据库系统概论（第 5 版）[M]. 北京: 高等教育出版社, 2014.

[40] Ronald Anjard. The Basics of Database Management Systems[J]. 1996.

[41] 朱少民. 软件测试方法和技术[M]. 3 版. 北京: 清华大学出版社, 2014.

[42] 佛罗桑. 软件工程：实践者的研究方法[M]. 8 版. 北京: 机械工业出版社, 2015.

[43] 贺平. 软件测试教程[M]. 3 版. 北京：电子工业出版社，2018.

[44] Pressman R S. 软件工程：实践者的研究方法[M]. 8 版. 北京：机械工业出版社，2016.